
mlquantify

Methods, Solvers, Representations & Losses

Comprehensive Documentation — API Docstrings & User Guide

Every quantifier, optimisation backend, feature representation and loss function, documented across the two-surface model.

Version 0.3.1

Author: Luiz Fernando Luth Junior

Theory grounded in the source papers (papers/Quantification & papers/Quantification/Methods)

How to read this document

This reference follows the **two-surface model** defined in the mlquantify documentation standard. Every entry is documented twice, for two different readers:

Surface	Audience	Content
API Docstring	Developer using the library	Interface: summary, parameters, attributes, examples, paper reference. No mathematics.
User Guide	Researcher learning the method	Theory: problem formulation, mathematical objective, algorithm, assumptions, relationships.

***The golden rule.** Mathematical formulations, derivations and pseudocode appear only in the User Guide blocks (shaded). The API Docstring blocks describe the interface in words. Throughout, p denotes a class-prevalence vector on the simplex ($p_i \geq 0$, $\sum p_i = 1$), a hat ($\hat{p}$ or p -hat) marks an estimate, 'tr' the training set and 'te' the test set.*

Each method also names the **distributional shift** it targets. Almost all assume **prior probability shift**: the class priors $p(y)$ change between training and test, while the class-conditional densities $p(x|y)$ stay fixed. This is the assumption under which the corrections below are unbiased.

PART I

Quantification Methods

The 33 quantifiers, organised by family. Each is documented as an API Docstring (interface, no math) followed by a User Guide (problem, objective, assumptions, relationships). The shift assumption and primary paper appear on both surfaces.

Counting

Counting methods estimate prevalence from the classifier's output on the test set and, except for the naive baselines, correct that count for the classifier's own error rates. They are the oldest and most widely used quantifier family.

CC — Classify & Count

Counting family | `from mlquantify.counting import CC`

API DOCSTRING · interface

Summary

Classify & Count (CC) quantifier.

Description

Targets prior probability shift. Classifies every test instance with a hard classifier and reports the fraction assigned to each class as the prevalence estimate. It is the simplest baseline and is systematically biased whenever the test class distribution differs from the training one.

Parameters

estimator	A classifier with <code>fit/predict</code> . If <code>None</code> , call <code>aggregate</code> with pre-computed labels.
threshold	float, default=0.5. Decision threshold applied to soft scores to form hard labels.

Attributes

estimator_	The fitted underlying classifier.
classes_	Class labels seen during fit.

Notes

CC does not correct for classifier bias, so its error grows roughly linearly as the test prevalence moves away from the training prevalence. Contrast with PCC (uses soft counts) and ACC (corrects for TPR/FPR).

See Also

PCC, ACC

Examples

```
>>> from mlquantify.counting import CC
>>> from sklearn.linear_model import LogisticRegression
>>> q = CC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)          # predict() path
{0: 0.47, 1: 0.53}
>>> preds = q.estimator_.predict(X_test)
>>> q.aggregate(preds, y_train=y_train)  # aggregate() path
{0: 0.47, 1: 0.53}
```

USER GUIDE · theory

Problem formulation

Let h be a classifier trained on a labelled sample. Under prior probability shift the densities $p(x|y)$ are shared by train and test while the priors $p(y)$ differ. The goal is to estimate the test priors $p_{te}(y)$ from an unlabelled test bag.

Algorithm / objective

CC takes the empirical distribution of hard predictions as the estimate:

$$\hat{p}(y) = (1/n) * \sum_{x \text{ in test}} I(h(x) = y)$$

No correction is applied: the raw classifier votes are simply counted and normalised.

Assumptions & when to use

Unbiased only when the classifier is perfect or when the test distribution equals the training distribution. Otherwise it inherits the classifier's confusion and under/over-estimates the minority class. Use it only as a baseline or when the classifier is extremely accurate.

Relationship to other methods

Root of the counting family. PCC replaces the hard vote $I(h(x)=y)$ with the posterior probability; ACC post-corrects CC using estimated TPR and FPR.

References

- [1] Forman, G. (2005). Counting Positives Accurately Despite Inaccurate Classification. *ECML*, 564–575.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

PCC — Probabilistic Classify & Count

Counting family | from `mlquantify.counting` import PCC

API DOCSTRING · interface

Summary

Probabilistic Classify & Count (PCC) quantifier.

Description

Targets prior probability shift. Averages the posterior probabilities returned by a probabilistic classifier over the test bag instead of counting hard labels. Generally less biased than CC but, lacking any correction, still drifts under strong shift and requires a calibrated classifier.

Parameters

estimator A classifier with `fit/predict_proba`. If None, call `aggregate` with posteriors.

Attributes

estimator_ The fitted underlying classifier.

classes_ Class labels seen during fit.

Notes

PCC is unbiased only if the posteriors are well calibrated on the test distribution; calibration drift re-introduces bias. It does not correct for classifier error.

See Also

CC, ACC, GPACC

Examples

```
>>> from mlquantify.counting import PCC
>>> q = PCC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.48, 1: 0.52}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.48, 1: 0.52}
```

USER GUIDE · theory

Problem formulation

Same prior-shift setting as CC, but the classifier exposes posteriors $P(y|x) = \hat{p}_{te}(y|x)$. PCC treats the mean posterior as the prevalence.

Algorithm / objective

Average the soft posteriors over the test bag:

$$\hat{p}_{\text{hat}}(y) = (1/n) * \sum_{\{x \text{ in test}\}} P(y | x)$$

Equivalent to CC when posteriors are hardened to 0/1, but soft counts reduce variance and exploit classifier confidence.

Assumptions & when to use

Unbiased under prior shift only with calibrated posteriors; otherwise biased like CC. Prefer it over CC whenever reliable probabilities are available and the shift is mild.

Relationship to other methods

Probabilistic counterpart of CC. GPACC is its multiclass, constrained-regression generalisation; ACC/PACC add an explicit error correction that PCC omits.

References

- [1] Bella, A., Ferri, C., Hernández-Orallo, J., & Ramírez-Quintana, M. J. (2010). Quantification via Probability Estimators. *ICDM*, 737–742.
- [2] Forman, G. (2005). Counting Positives Accurately Despite Inaccurate Classification. *ECML*, 564–575.

ACC — Adjusted Classify & Count

Counting family | from `mlquantify.counting` import `ACC`

API DOCSTRING · interface

Summary

Adjusted Classify & Count (ACC) quantifier.

Description

Targets prior probability shift. Runs CC and then corrects the biased count using the classifier's true- and false-positive rates, estimated by cross-validation on the training set. The correction is exact in expectation but becomes unstable when the rates are close together (low-separability classifiers).

Parameters

estimator	A classifier with <code>fit/predict</code> .
threshold	float, default=0.5. Operating point at which TPR and FPR are measured.
cv	int, cross-validation folds used to estimate TPR and FPR.

Attributes

estimator_	The fitted underlying classifier.
tpr_	Estimated true-positive rate at the chosen threshold.
fpr_	Estimated false-positive rate at the chosen threshold.
classes_	Class labels seen during fit.

Notes

ACC corrects the linear bias of CC and is unbiased when the estimated TPR/FPR match the test set. The estimate is clipped to $[0,1]$; the denominator ($TPR-FPR$) controls its variance.

See Also

CC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import ACC
>>> q = ACC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.30, 1: 0.70}
>>> preds = q.estimator_.predict(X_test)
>>> q.aggregate(preds, y_train=y_train)
{0: 0.30, 1: 0.70}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. The observed positive count produced by the classifier mixes true positives and false positives at the (shift-invariant) rates TPR and FPR.

Algorithm / objective

The observed positive rate decomposes as a linear function of the true prevalence p :

$$\begin{aligned} CC &= TPR * p + FPR * (1 - p) \\ &= (TPR - FPR) * p + FPR \\ \Rightarrow p_hat &= (CC - FPR) / (TPR - FPR), \text{ clipped to } [0, 1] \end{aligned}$$

TPR and FPR are estimated once by cross-validation on the training data and reused for any test bag. Multiclass uses the analogous misclassification-matrix inversion.

Assumptions & when to use

Unbiased when TPR/FPR are stable across the shift (prior-shift assumption). When TPR–FPR is small — a weak classifier under heavy imbalance — the division amplifies estimation error, so ACC trades CC's bias for variance. Best with a reasonably separating classifier.

Relationship to other methods

Adds the TPR/FPR correction missing from CC. The threshold-policy methods (TAC, TX, TMAX, T50, MS, MS2) are ACC evaluated at cleverly chosen operating points.

References

- [1] Forman, G. (2005). Counting Positives Accurately Despite Inaccurate Classification. *ECML*, 564–575.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

TAC — Threshold Adjusted Count

Counting family | from `mlquantify.counting` import TAC

API DOCSTRING · interface

Summary

Threshold Adjusted Count (TAC) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. TAC applies the correction at the default decision threshold. Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

Equivalent to binary ACC at threshold 0.5; the simplest threshold policy.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples


```
>>> from mlquantify.counting import TAC
>>> q = TAC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[:, 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated TPR(t)/FPR(t) curves.

Algorithm / objective

TAC fixes the operating point at the classifier's default threshold (0.5) and corrects there:

```
t* = 0.5
p_hat = ( CC(t*) - FPR(t*) ) / ( TPR(t*) - FPR(t*) )
```

With the threshold fixed, the standard adjusted count $p_hat = (CC - FPR)/(TPR - FPR)$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

TX — Threshold at the X-crossing

Counting family | from `mlquantify.counting` import TX

API DOCSTRING · interface

Summary

Threshold at the X-crossing (TX) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. TX (the 'X method') picks the threshold where FPR(t) crosses 1–TPR(t). Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
-------------------	-----------------------------------

threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

The crossing point keeps both error rates moderate and TPR–FPR comfortably non-zero, giving a stable denominator under imbalance.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import TX
>>> q = TX(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated TPR(t)/FPR(t) curves.

Algorithm / objective

TX chooses the operating point where the false-positive rate equals the false-negative rate, i.e. where the two error curves cross:

$$t^* = \operatorname{argmin}_t | \operatorname{FPR}(t) - (1 - \operatorname{TPR}(t)) |$$

(equivalently $\operatorname{TPR}(t^*) = 1 - \operatorname{FPR}(t^*)$)

With the threshold fixed, the standard adjusted count $p_{\text{hat}} = (\text{CC} - \text{FPR})/(\text{TPR} - \text{FPR})$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

TMAX — Threshold at maximum TPR–FPR

Counting family | from `mlquantify.counting` import TMAX

API DOCSTRING · interface

Summary

Threshold at maximum TPR–FPR (TMAX) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. TMAX (the 'Max method') maximises the gap $\text{TPR}(t) - \text{FPR}(t)$. Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

Maximising TPR–FPR yields the most stable denominator, but Forman reports TMAX carries a systematic linear bias because the chosen rates often fail to transfer to the test set.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import TMAX
>>> q = TMAX(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated $\text{TPR}(t)/\text{FPR}(t)$ curves.

Algorithm / objective

TMAX selects the threshold that maximises classifier separation (the Youden index); ties are resolved at the midpoint of the maximal range:

$$t^* = \operatorname{argmax}_t (\text{TPR}(t) - \text{FPR}(t))$$

With the threshold fixed, the standard adjusted count $p_{\text{hat}} = (\text{CC} - \text{FPR})/(\text{TPR} - \text{FPR})$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

T50 — Threshold at TPR = 0.5

Counting family | from `mlquantify.counting` import T50

API DOCSTRING · interface

Summary

Threshold at TPR = 0.5 (T50) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. T50 pins the threshold where the true-positive rate equals 0.5. Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

Fixing TPR makes the policy insensitive to positive-class scarcity; robust with as few as ~10 training positives, per Forman.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import T50
>>> q = T50(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated TPR(t)/FPR(t) curves.

Algorithm / objective

T50 chooses the operating point at which exactly half of the positives are recovered:

$$t^* = \operatorname{argmin}_t | \operatorname{TPR}(t) - 0.5 |$$

With the threshold fixed, the standard adjusted count $p_{\text{hat}} = (\text{CC} - \text{FPR})/(\text{TPR} - \text{FPR})$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

MS — Median Sweep

Counting family | from `mlquantify.counting` import MS

API DOCSTRING · interface

Summary

Median Sweep (MS) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. MS computes an adjusted count at every threshold and returns the median. Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

The median is robust to the unreliable thresholds (small TPR–FPR) that would wreck a single-point ACC; Forman found Median Sweep outstandingly robust across shifts.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import MS
>>> q = MS(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[:, 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated TPR(t)/FPR(t) curves.

Algorithm / objective

Rather than trusting a single operating point, MS sweeps all thresholds, computes one adjusted count per threshold, and reports the median estimate:

```
for each threshold t:  p_t = ( CC(t) - FPR(t) ) / ( TPR(t) - FPR(t) )
p_hat = median_t  p_t
```

With the threshold fixed, the standard adjusted count $p_{\text{hat}} = (CC - FPR)/(TPR - FPR)$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

MS2 — Median Sweep (restricted)

Counting family | from `mlquantify.counting` import MS2

API DOCSTRING · interface

Summary

Median Sweep (restricted) (MS2) quantifier.

Description

Targets prior probability shift. A threshold-policy member of the Adjusted Count family: it selects a classification threshold by a fixed rule, measures TPR and FPR there by cross-validation, and applies the adjusted-count correction. MS2 is Median Sweep restricted to thresholds with a reliable denominator. Binary-only.

Parameters

estimator	A classifier exposing soft scores (<code>predict_proba/decision_function</code>).
cv	int, cross-validation folds used to build the threshold–rate table.

Attributes

estimator_	The fitted underlying classifier.
threshold_	The operating threshold chosen by the policy.
tpr_	True-positive rate at the chosen threshold(s).
fpr_	False-positive rate at the chosen threshold(s).
classes_	Class labels seen during fit.

Notes

By pruning low-separation thresholds MS2 removes the highest-variance terms of the sweep, usually improving on MS when the classifier is weak in part of the score range.

See Also

ACC, TAC, TX, TMAX, T50, MS, MS2

Examples

```
>>> from mlquantify.counting import MS2
>>> q = MS2(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.41, 1: 0.59}
>>> scores = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(scores, y_train=y_train)
{0: 0.41, 1: 0.59}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. ACC's correction is exact only if TPR–FPR is well away from zero; the choice of operating threshold therefore controls the estimator's stability. Each policy picks that threshold differently from the cross-validated TPR(t)/FPR(t) curves.

Algorithm / objective

MS2 runs the same sweep but discards operating points where TPR(t)–FPR(t) is too small (below 0.25) before taking the median:

$$S = \{ t : (\text{TPR}(t) - \text{FPR}(t)) > 0.25 \}$$

$$p_{\text{hat}} = \text{median}_{\{t \text{ in } S\}} (\text{CC}(t) - \text{FPR}(t)) / (\text{TPR}(t) - \text{FPR}(t))$$

With the threshold fixed, the standard adjusted count $p_{\text{hat}} = (\text{CC} - \text{FPR})/(\text{TPR} - \text{FPR})$ is applied at that point.

Assumptions & when to use

Same prior-shift assumption as ACC. The policy aims to keep TPR–FPR large (reliable denominator). Threshold rules that pin a specific rate (T50, TX) are robust under imbalance; TMAX maximises separation but can carry a systematic bias.

Relationship to other methods

All are ACC at a chosen operating point. MS/MS2 aggregate over many thresholds instead of committing to one, trading a little bias for much lower variance.

References

- [1] Forman, G. (2006). Quantifying Trends Accurately Despite Classifier Error and Class Imbalance. *KDD'06*, 157–166.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

GACC — Generalized Adjusted Count

Counting family | from `mlquantify.counting` import `GACC`

API DOCSTRING · interface

Summary

Generalized Adjusted Count (GACC) quantifier.

Description

Targets prior probability shift. A multiclass quantifier built on the unified constrained-regression framework: it summarises train and test through a feature transform and solves for the prevalence vector on the simplex. GACC transforms each instance into its hard one-hot predicted class and matches the resulting class-frequency vectors. Native multiclass (no One-vs-All).

Parameters

estimator	A probabilistic classifier.
solver	Simplex optimiser, default 'slsqp' (see solvers).
cv	int, cross-validation folds used to build the per-class transform matrix.

Attributes

estimator_	The fitted underlying classifier.
representation_	The fitted feature transform (per-class matrix X).
classes_	Class labels seen during fit.

Notes

Multiclass generalisation of ACC: the per-class matrix X is the training confusion matrix, and solving $Xp=y$ inverts it on the simplex. Solved on the probability simplex ($p \geq 0$, $\sum p = 1$), so the estimate is always a valid distribution.

See Also

GACC, GPACC, FM, GHDy, GHDx

Examples

```
>>> from mlquantify.counting import GACC
>>> q = GACC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)          # multiclass
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Firat (2016) unifies quantifiers as a constrained multivariate regression. With a feature transform $f(x)$, let X be the matrix whose column k is the per-class mean transform and y the mean transform on the test bag. Under prior shift the test transform is the prevalence-weighted mixture of the per-class transforms:

Algorithm / objective

Generalized Adjusted Count uses the transform: $f(x) = \text{onehot}(\text{argmax}_k P(k|x))$ (hard class assignment). The prevalence solves the constrained regression

```
minimize_p    L( y , X p )
subject to    p >= 0 ,    sum_k p_k = 1

L( y, Xp ) = || y - X p ||^2      (least-squares / L2)
```


X is estimated once by cross-validation on the training set; the same X serves any test bag. Because the model is multivariate, the binary method extends to multiclass automatically.

Assumptions & when to use

Requires that the chosen transform's class-conditional distribution is invariant across the shift (an extension of prior-shift to transform space) and that the per-class transforms are sufficiently distinct (identifiability). Best for multiclass problems where One-vs-All counting breaks down.

Relationship to other methods

Members of one framework differing only in transform and loss: GACC uses hard one-hot counts, GPACC uses soft posteriors, FM uses Friedman's prior-threshold indicator, GHDy/GHDx use histograms with a Hellinger loss.

References

- [1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.
- [2] Forman, G. (2008). Quantifying Counts and Costs via Classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206.

GPACC — Generalized Probabilistic Adjusted Count

Counting family | from `mlquantify.counting` import GPACC

API DOCSTRING · interface

Summary

Generalized Probabilistic Adjusted Count (GPACC) quantifier.

Description

Targets prior probability shift. A multiclass quantifier built on the unified constrained-regression framework: it summarises train and test through a feature transform and solves for the prevalence vector on the simplex. GPACC transforms each instance into its soft posterior vector and matches the mean posteriors. Requires soft posteriors. Native multiclass (no One-vs-All).

Parameters

estimator	A probabilistic classifier (uses <code>predict_proba</code>).
solver	Simplex optimiser, default 'slsqp' (see solvers).
cv	int, cross-validation folds used to build the per-class transform matrix.

Attributes

estimator_	The fitted underlying classifier.
representation_	The fitted feature transform (per-class matrix X).
classes_	Class labels seen during fit.

Notes

Multiclass, corrected generalisation of PCC (a.k.a. PACC): the soft-count matrix is inverted on the simplex instead of being read off directly. Solved on the probability simplex ($p \geq 0$, $\sum p = 1$), so the estimate is always a valid distribution.

See Also

GACC, GPACC, FM, GHDy, GHDx

Examples

```
>>> from mlquantify.counting import GPACC
>>> q = GPACC(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)           # multiclass
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Firat (2016) unifies quantifiers as a constrained multivariate regression. With a feature transform $f(x)$, let X be the matrix whose column k is the per-class mean transform and y the mean transform on the test bag. Under prior shift the test transform is the prevalence-weighted mixture of the per-class transforms:

Algorithm / objective

Generalized Probabilistic Adjusted Count uses the transform: $f(x) = P(\cdot|x)$ (soft posterior vector). The prevalence solves the constrained regression

```
minimize_p    L( y ,  X p )
subject to    p >= 0 ,    sum_k p_k = 1

L( y, Xp ) = || y - X p ||^2      (least-squares / L2)
```

X is estimated once by cross-validation on the training set; the same X serves any test bag. Because the model is multivariate, the binary method extends to multiclass automatically.

Assumptions & when to use

Requires that the chosen transform's class-conditional distribution is invariant across the shift (an extension of prior-shift to transform space) and that the per-class transforms are sufficiently distinct (identifiability). Best for multiclass problems where One-vs-All counting breaks down.

Relationship to other methods

Members of one framework differing only in transform and loss: GACC uses hard one-hot counts, GPACC uses soft posteriors, FM uses Friedman's prior-threshold indicator, GHDy/GHDx use histograms with a Hellinger loss.

References

- [1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.
- [1] Bella, A., Ferri, C., Hernández-Orallo, J., & Ramírez-Quintana, M. J. (2010). Quantification via Probability Estimators. *ICDM*, 737–742.

FM — Friedman's Method

Counting family | from `mlquantify.counting` import FM

API DOCSTRING · interface

Summary

Friedman's Method (FM) quantifier.

Description

Targets prior probability shift. A multiclass quantifier built on the unified constrained-regression framework: it summarises train and test through a feature transform and solves for the prevalence vector on the simplex. FM transforms each instance with an indicator of whether its posterior for class k exceeds the training prior of class k . Requires soft posteriors. Native multiclass (no One-vs-All).

Parameters

estimator	A probabilistic classifier (uses <code>predict_proba</code>).
solver	Simplex optimiser, default <code>'slsqp'</code> (see solvers).
cv	int, cross-validation folds used to build the per-class transform matrix.

Attributes

estimator_	The fitted underlying classifier.
representation_	The fitted feature transform (per-class matrix X).
classes_	Class labels seen during fit.

Notes

Friedman's dynamic-threshold quantifier: thresholding the posterior at the class's own training prior minimises the variance of the proportion estimate. Solved on the probability simplex ($p \geq 0$, $\sum p = 1$), so the estimate is always a valid distribution.

See Also

GACC, GPACC, FM, GHDy, GHDx

Examples

```
>>> from mlquantify.counting import FM
>>> q = FM(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)          # multiclass
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Firat (2016) unifies quantifiers as a constrained multivariate regression. With a feature transform $f(x)$, let X be the matrix whose column k is the per-class mean transform and y the mean transform on the test bag. Under prior shift the test transform is the prevalence-weighted mixture of the per-class transforms:

Algorithm / objective

Friedman's Method uses the transform: $f_k(x) = I(P(k|x) \geq \text{prior_tr}(k))$ (Friedman threshold = training prior). The prevalence solves the constrained regression

```
minimize_p    L( y , X p )
subject to    p >= 0 ,    sum_k p_k = 1

L( y, Xp ) = || y - X p ||^2    (least-squares / L2)
```

X is estimated once by cross-validation on the training set; the same X serves any test bag. Because the model is multivariate, the binary method extends to multiclass automatically.

Assumptions & when to use

Requires that the chosen transform's class-conditional distribution is invariant across the shift (an extension of prior-shift to transform space) and that the per-class transforms are sufficiently distinct (identifiability). Best for multiclass problems where One-vs-All counting breaks down.

Relationship to other methods

Members of one framework differing only in transform and loss: GACC uses hard one-hot counts, GPACC uses soft posteriors, FM uses Friedman's prior-threshold indicator, GHDy/GHDx use histograms with a Hellinger loss.

References

- [1] Friedman, J. (2014). Class Counts in Future Unlabeled Samples (Detecting and Dealing with Concept Drift). *MIT CSAIL Big Data Event*.
- [1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.

Matching

Distribution-matching (mixture-model) methods model the test sample as a prevalence-weighted mixture of the per-class distributions and search for the mixing weights that make the mixture most similar to the observed test distribution. They differ in how a distribution is represented (histogram, kernel, raw sample, embedding) and in the divergence minimised.

DyS — Distribution y-Similarity

Matching family | from `mlquantify.matching` import `DyS`

API DOCSTRING · interface

Summary

Distribution y-Similarity (DyS) quantifier.

Description

Targets prior probability shift. A general mixture-model framework: it builds score histograms for each class, mixes them by a candidate prevalence, and searches for the mixture that minimises a chosen histogram dissimilarity against the test histogram. HDy is a special case; Topsoe is the recommended distance. Binary base method.

Parameters

estimator	A probabilistic classifier (positive-class scores).
measure	Histogram dissimilarity minimised between the mixed and test histograms; default 'topsoe'. • topsoe — symmetric information-theoretic distance; most accurate when bins are tuned (recommended). • hellinger — bounded distance over sqrt-probabilities; the classic HDy choice. • prob_symm — probabilistic symmetric chi-square distance. • sqEuclidean — squared Euclidean distance between the bin vectors.
bins_size	array of bin counts to sweep; controls histogram resolution. Smaller counts ($\leq \sim 20$) usually work best; estimates are aggregated by their median.
solver	Scalar search strategy over the prevalence alpha. • ternary — trisection search; fast, exploits the near-unimodal objective (default). • grid — exhaustive search over an evenly-spaced grid of alpha values. • bounded — scipy bounded scalar minimiser.

Attributes

estimator_	The fitted underlying classifier.
pos_scores_	Cross-validated positive-class score sample.
neg_scores_	Cross-validated negative-class score sample.
classes_	Class labels seen during fit.

Notes

Best bin counts are small (typically ≤ 20), contradicting HDy's original 10–110 range; the per-bin estimates are aggregated by their median. Ternary search suffices because the objective is near-unimodal in alpha.

See Also

HDy, SORD, SMM, HDx

Examples

```
>>> from mlquantify.matching import DyS
>>> q = DyS(LogisticRegression(), measure='topsoe').fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.49, 1: 0.51}
>>> ts = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(ts)
{0: 0.48, 1: 0.52}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. Let $H(S)$ be the score histogram of a sample S . The positive training scores S^+ and negative scores S^- give class histograms; the test scores Z give the observed histogram. The test histogram is assumed to be the alpha-mixture of the class histograms, alpha being the positive prevalence.

Algorithm / objective

Search the mixing weight alpha that minimises a dissimilarity DS between the mixed class histogram and the test histogram:

```
alpha* = argmin_{0 <= alpha <= 1}
        DS( alpha*H(S+) + (1-alpha)*H(S-) , H(Z) )
p_hat = [ 1 - alpha* , alpha* ]
```

The search uses ternary search; the procedure is repeated for several bin counts and the median alpha is returned. With $DS = \text{Hellinger}$ this reduces exactly to HDy .

Assumptions & when to use

Needs scores whose class-conditional distribution is shift-invariant, and a test bag large enough to estimate $H(Z)$. Robust under imbalance; the parameter-sensitive choice is the bin count, which is why SORD/SMM (bin-free members) exist. Strong general-purpose binary quantifier.

Relationship to other methods

Generalises HDy (Hellinger) to any symmetric distance. SORD and SMM are DyS members that drop histograms (operate on the raw score sample and the score mean).

References

- [1] Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS : A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.
- [2] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

HDy — Hellinger Distance y

Matching family | `from mlquantify.matching import HDy`

API DOCSTRING · interface

Summary

Hellinger Distance y (HDy) quantifier.

Description

Targets prior probability shift. Represents each class by a histogram of classifier scores and finds the positive prevalence whose mixed histogram minimises the Hellinger distance to the test histogram. The original mixture-model quantifier on posteriors; binary base method run over a range of bin counts.

Parameters

estimator	A probabilistic classifier (positive-class scores).
bins_size	array of bin counts to sweep; controls histogram resolution. Estimates over the different bin counts are aggregated by their median.

solver	Scalar search strategy over the positive prevalence alpha. - linear — scan alpha on an evenly-spaced grid; the original HDy search. - ternary — interval-trisection search; faster and usually as precise.
Attributes	
estimator_	The fitted underlying classifier.
pos_scores_	Cross-validated positive-class score sample.
neg_scores_	Cross-validated negative-class score sample.
classes_	Class labels seen during fit.

Notes

Estimates are computed for several bin counts and aggregated by the median. HDy is the Hellinger instance of DyS; switching to Topsoe (DyS) often lowers error.

See Also

DyS, HDx, GHDy

Examples

```
>>> from mlquantify.matching import HDy
>>> q = HDy(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.49, 1: 0.51}
>>> ts = q.estimator_.predict_proba(X_test)[: , 1]
>>> q.aggregate(ts)
{0: 0.49, 1: 0.51}
```

USER GUIDE · theory**Problem formulation**

Binary prior shift with class score histograms P^+ , P^- and test histogram Q , each a normalised vector over b bins.

Algorithm / objective

Minimise the Hellinger distance between the mixed histogram and the test histogram over the positive prevalence alpha:

$$\begin{aligned} \text{HD}(P, Q) &= \sqrt{1 - \sum_i \sqrt{P_i \cdot Q_i}} \\ \alpha^* &= \operatorname{argmin}_{\{0 \leq \alpha \leq 1\}} \\ &\quad \text{HD}(\alpha P^+ + (1-\alpha)P^-, Q) \end{aligned}$$

Repeated for bins 10,20,...,110 in the original paper; the final estimate is the median over bin counts. mlquantify also offers Laplace-smoothed histograms.

Assumptions & when to use

Same prior-shift / shift-invariant-score assumption as DyS. Sensitive to the bin count when scores are sparse; the median over bins mitigates this. A robust, well-tested binary default.

Relationship to other methods

The canonical mixture model; DyS generalises its distance, HDx moves the matching into feature space, and GHDy lifts it to multiclass constrained regression.

References

- [1] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.
- [2] Maletke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.

HDx — Hellinger Distance x

Matching family | from mlquantify.matching import HDx

API DOCSTRING · interface

Summary

Hellinger Distance x (HDx) quantifier.

Description

Targets prior probability shift. The classifier-free mixture model: it builds a histogram for every input feature and matches the prevalence-mixed per-feature histograms to the test histograms under the Hellinger distance. Needs no scorer, only the raw features. Binary base method.

Parameters

bins_size	array of bin counts to sweep per feature.
solver	Scalar search over alpha.

Attributes

pos_repr_	Per-feature histograms of the positive class.
neg_repr_	Per-feature histograms of the negative class.
classes_	Class labels seen during fit.

Notes

Because it skips the classifier, HDx avoids calibration issues but cannot exploit a good scorer; per-feature distances are aggregated (median) into one estimate.

See Also

HDy, GHDx

Examples

```
>>> from mlquantify.matching import HDx
>>> q = HDx().fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.52, 1: 0.48}
>>> q.aggregate(X_test)      # features, no classifier
{0: 0.52, 1: 0.48}
```

USER GUIDE · theory

Problem formulation

Binary prior shift matched directly in feature space: each feature j has class histograms P^{+j} , P^{-j} and test histogram Q^j .

Algorithm / objective

For each feature find the alpha minimising the Hellinger distance, then aggregate across features:

```
alpha_j* = argmin_alpha HD( alpha*P^{+j} + (1-alpha)*P^{-j} , Q^j )
p_hat(+) = median_j alpha_j*
```

Sweeping bin counts and taking medians (over bins and features) yields the final estimate. No cross-validation is needed since there is no classifier.

Assumptions & when to use

Assumes each feature's class-conditional distribution is shift-invariant and informative. Works without a trained model, useful when a reliable classifier is unavailable, but degrades with many weak/correlated features.

Relationship to other methods

Feature-space twin of HDy. GHDx is its multiclass constrained-regression form.

References

[1] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

SORD — Sample Ordinal Distance

Matching family | from mlquantify.matching import SORD

API DOCSTRING · interface

Summary

Sample Ordinal Distance (SORD) quantifier.

Description

Targets prior probability shift. A parameter-free mixture model: instead of binning scores it compares the raw weighted score samples with an ordinal (earth-mover-like) distance, searching the prevalence that best mixes the positive and negative score samples to match the test sample. Binary base method, no bins to tune.

Parameters

estimator	A probabilistic classifier (positive-class scores).
solver	Scalar search over alpha (ternary search).

Attributes

estimator_	The fitted underlying classifier.
pos_scores_	Cross-validated positive-class score sample.
neg_scores_	Cross-validated negative-class score sample.
classes_	Class labels seen during fit.

Notes

Parameter-free (no bin count). Cost grows as $O(n \log n)$ in the combined sample size, so training scores are sub-sampled (~1000/class). Competitive with a tuned Topsoe-DyS.

See Also

DyS, SMM, HDy

Examples

```
>>> from mlquantify.matching import SORD
>>> q = SORD(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.50, 1: 0.50}
>>> ts = q.estimator_.predict_proba(X_test)[:, 1]
>>> q.aggregate(ts)
{0: 0.50, 1: 0.50}
```

USER GUIDE · theory

Problem formulation

Binary prior shift compared on raw score samples. Each observation is given unit mass scaled by its sample size so the mixed training sample and the test sample carry equal total mass; SORD is the minimum cost to morph one weighted sample into the other (a 1-D earth-mover / MDPA distance, the bins-to-infinity limit of ordinal distance).

Algorithm / objective

Assign signed weights to the pooled, sorted scores and accumulate the absolute running mass times the gaps:


```

w(x) = +alpha/|S+|    if x in S+
      +(1-alpha)/|S-| if x in S-
      -1/|Z|          if x in Z    (test)
v = sort( S+  U  S-  U  Z )
SORD(alpha) = sum_i | (v_i - v_{i-1}) * cumsum(w)_i |
alpha* = argmin_alpha SORD(alpha)

```

Ternary search over alpha; no histogram and therefore no bin parameter.

Assumptions & when to use

Same shift-invariant-score assumption as DyS, but immune to bin mis-specification and the curse of dimensionality in score space. Use it when you want a robust, tuning-free binary quantifier and can afford the slightly higher compute.

Relationship to other methods

A bin-free member of the DyS framework ($H(x)=x$). Where SMM matches only the mean of the score sample, SORD matches the whole ordinal shape.

References

[1] Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.

SMM — Sample Mean Matching

Matching family | from `mlquantify.matching` import SMM

API DOCSTRING · interface

Summary

Sample Mean Matching (SMM) quantifier.

Description

Targets prior probability shift. The lightest mixture model: it summarises each class by its mean classifier score and solves in closed form for the prevalence that makes the mixed mean equal the test mean score. Binary base method, no search and no bins.

Parameters

estimator A probabilistic classifier (positive-class scores).

Attributes

estimator_	The fitted underlying classifier.
pos_mean_	Mean positive-class score (cross-validated).
neg_mean_	Mean negative-class score (cross-validated).
classes_	Class labels seen during fit.

Notes

Closed-form and extremely fast, but uses only the first moment of the score distribution, so it is less accurate than DyS/SORD when class scores overlap.

See Also

DyS, SORD

Examples

```
>>> from mlquantify.matching import SMM
>>> q = SMM(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.46, 1: 0.54}
>>> ts = q.estimator_.predict_proba(X_test)[:, 1]
>>> q.aggregate(ts)
{0: 0.46, 1: 0.54}
```

USER GUIDE · theory

Problem formulation

Binary prior shift matched on the mean score. With mean positive score μ_+ , mean negative score μ_- and mean test score μ_Z , the test mean is the alpha-mixture of the class means.

Algorithm / objective

Solve the one-equation mean-matching condition directly:

$$\alpha * \mu_+ + (1 - \alpha) * \mu_- = \mu_Z$$

$$\alpha^* = (\mu_Z - \mu_-) / (\mu_+ - \mu_-) \quad (\text{clipped to } [0,1])$$

No optimisation loop; a single division gives the estimate.

Assumptions & when to use

Exact only if the class score distributions differ enough in mean and are shift-invariant. Degenerates when $\mu_+ \approx \mu_-$ (overlapping classes). Best as a fast baseline or when scores are well separated.

Relationship to other methods

The first-moment special case of DyS/SORD: matching means instead of full histograms or ordinal shapes.

References

[1] Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.

MMD_RKHS — Maximum Mean Discrepancy (RKHS)

Matching family | from mlquantify.matching import MMD_RKHS

API DOCSTRING · interface

Summary

Maximum Mean Discrepancy in an RKHS (MMD_RKHS) quantifier.

Description

Targets prior probability shift. Matches distributions by their mean embeddings in a reproducing-kernel Hilbert space: it finds the prevalence vector whose mixture of per-class mean embeddings is closest to the test mean embedding. Kernel-based, multiclass, provably consistent, and free of density or histogram estimation.

Parameters

kernel	Kernel defining the RKHS feature map; must be universal for consistency. Default 'rbf'. - rbf — Gaussian radial-basis kernel; universal, the default choice. - linear — plain inner product; fast but only matches first moments. - poly — polynomial kernel of a given degree; matches higher moments.
gamma	Kernel bandwidth (RBF/poly). Small gamma over-smooths; large gamma over-fits the embedding. None uses $1/n_{\text{features}}$.
solver	Simplex optimiser for the convex QP, default 'lsqp'.

Attributes

representation_	Fitted kernel-mean embeddings per class.
------------------------	--

classes_ Class labels seen during fit.

Notes

Statistically consistent under a universal kernel; its error shrinks when classes are well separated (large minimum eigenvalue) and the data spread is small. Kernel choice matters and can be tuned.

See Also

EDy, KDEyML

Examples

```
>>> from mlquantify.matching import MMD_RKHS
>>> q = MMD_RKHS(kernel='rbf').fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.33, 1: 0.33, 2: 0.34}
>>> q.aggregate(X_test)
{0: 0.33, 1: 0.33, 2: 0.34}
```

USER GUIDE · theory

Problem formulation

Prior shift with a universal kernel K and feature map Φ . The test distribution is the prevalence-weighted mixture of the class-conditional distributions; matching is done between their mean embeddings $\Phi_y = E[\Phi(x)|y]$ and the test mean Φ_U .

Algorithm / objective

Minimise the squared RKHS distance between the mixed and test mean embeddings — a convex quadratic program on the simplex:

```
minimize_theta || sum_y theta_y * Phi_y - Phi_U ||^2
subject to      theta >= 0 , sum_y theta_y = 1
kernelised:    theta^T (A^T A) theta - 2 theta^T (A^T a)
               A = [Phi_1-Phi_0, ...] , a = Phi_U - Phi_0 (inner products via K)
```

All inner products are computed with the kernel trick; the QP is solved on the simplex. Estimator-free — it embeds features directly.

Assumptions & when to use

Consistent when K is universal and the class mixtures are identifiable (distinct embeddings). Needs enough test points to estimate Φ_U . Strong choice for multiclass and high-dimensional data; sensitive to kernel bandwidth.

Relationship to other methods

Like EDy/EDx it matches whole samples rather than histograms, but in an RKHS via kernel mean embeddings; EDy uses an energy distance instead of a kernel norm.

References

[1] Iyer, A., Nath, S., & Sarawagi, S. (2014). Maximum Mean Discrepancy for Class Ratio Estimation: Convergence Bounds and Kernel Selection. *ICML*, 32.

KDEyML — Kernel Density Estimation y — Maximum Likelihood

Matching family | from mlquantify.matching import KDEyML

API DOCSTRING · interface

Summary

Kernel Density Estimation y — Maximum Likelihood (KDEyML) quantifier.

Description

Targets prior probability shift. A multiclass distribution-matching method that replaces the per-class histograms of HDy with a single multivariate kernel density estimate over the posterior vectors on the

simplex. KDEyML chooses the prevalence by maximum likelihood of the test posteriors under the mixture of per-class KDEs. Native multiclass; scales to many classes where histogram matching fragments.

Parameters

estimator	A probabilistic classifier (posterior vectors).
bandwidth	float, KDE bandwidth; controls density smoothness.
solver	Simplex optimiser / search used to minimise the objective.

Attributes

estimator_	The fitted underlying classifier.
representation_	Per-class fitted KDE models.
classes_	Class labels seen during fit.

Notes

Likelihood objective; often beats EM (EMQ) when posteriors are recalibrated. Equivalent in spirit to EMQ but with KDE-smoothed class densities.

See Also

KDEyML, KDEyHD, KDEyCS, GKDEyML

Examples

```
>>> from mlquantify.matching import KDEyML
>>> q = KDEyML(LogisticRegression(), bandwidth=0.1).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Prior shift on the $(n-1)$ -simplex of posterior vectors. Each class i is modelled by a density p_i fitted with KDE; the test posteriors are assumed drawn from the prevalence mixture $\sum_i p_{i_prev} * p_i$. KDE replaces histograms so the multivariate simplex is not fragmented into bins.

Algorithm / objective

Maximise the log-likelihood of the test posteriors under the KDE mixture:

$$\begin{aligned} &\text{maximize}_p \quad \sum_{x \in \text{test}} \log(\sum_i p_i * \text{KDE}_i(x)) \\ &\text{subject to} \quad p \geq 0, \quad \sum_i p_i = 1 \end{aligned}$$

Per-class KDEs are fitted once at training time; only the objective changes between the ML, HD and CS variants.

Assumptions & when to use

Assumes posteriors are shift-invariant per class and that the bandwidth is well chosen (over-smoothing flattens class differences). Excels at multiclass quantification with moderate-to-many classes; KDEyML benefits from recalibrated posteriors.

Relationship to other methods

The KDE generalisation of histogram matching (HDy/DyS). KDEyML is a likelihood method (compare EMQ); KDEyHD/KDEyCS are distribution-matching with Hellinger / Cauchy-Schwarz divergences.

References

[1] Moreo, A., González, P., & del Coz, J. J. (2024). Kernel Density Estimation for Multiclass Quantification. *arXiv:2401.00490*.

KDEyHD — Kernel Density Estimation y — Hellinger

Matching family | from mlquantify.matching import KDEyHD

API DOCSTRING · interface

Summary

Kernel Density Estimation y — Hellinger (KDEyHD) quantifier.

Description

Targets prior probability shift. A multiclass distribution-matching method that replaces the per-class histograms of HDy with a single multivariate kernel density estimate over the posterior vectors on the simplex. KDEyHD minimises the Hellinger distance between the test KDE and the mixture KDE, estimated by Monte-Carlo sampling. Native multiclass; scales to many classes where histogram matching fragments.

Parameters

estimator	A probabilistic classifier (posterior vectors).
bandwidth	float, KDE bandwidth; controls density smoothness.
solver	Simplex optimiser / search used to minimise the objective.

Attributes

estimator_	The fitted underlying classifier.
representation_	Per-class fitted KDE models.
classes_	Class labels seen during fit.

Notes

Distribution-matching analogue of HDy lifted to multivariate KDEs; the integral is approximated by sampling points from the fitted mixture.

See Also

KDEyML, KDEyHD, KDEyCS, GKDEyML

Examples

```
>>> from mlquantify.matching import KDEyHD
>>> q = KDEyHD(LogisticRegression(), bandwidth=0.1).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Prior shift on the $(n-1)$ -simplex of posterior vectors. Each class i is modelled by a density p_i fitted with KDE; the test posteriors are assumed drawn from the prevalence mixture $\sum_i p_{i_prev} * p_i$. KDE replaces histograms so the multivariate simplex is not fragmented into bins.

Algorithm / objective

Minimise the (Monte-Carlo estimated) Hellinger divergence between the mixture density and the test density:

```
minimize_p    HD( sum_i p_i * KDE_i , KDE_test )
              HD^2(f,g) = 1 - integral sqrt( f * g )    (Monte-Carlo over simplex samples)
subject to    p >= 0 , sum_i p_i = 1
```

Per-class KDEs are fitted once at training time; only the objective changes between the ML, HD and CS variants.

Assumptions & when to use

Assumes posteriors are shift-invariant per class and that the bandwidth is well chosen (over-smoothing flattens class differences). Excels at multiclass quantification with moderate-to-many classes; KDEyML benefits from recalibrated posteriors.

Relationship to other methods

The KDE generalisation of histogram matching (HDy/DyS). KDEyML is a likelihood method (compare EMQ); KDEyHD/KDEyCS are distribution-matching with Hellinger / Cauchy-Schwarz divergences.

References

[1] Moreo, A., González, P., & del Coz, J. J. (2024). Kernel Density Estimation for Multiclass Quantification. *arXiv:2401.00490*.

KDEyCS — Kernel Density Estimation y — Cauchy-Schwarz

Matching family | from mlquantify.matching import KDEyCS

API DOCSTRING · interface

Summary

Kernel Density Estimation y — Cauchy-Schwarz (KDEyCS) quantifier.

Description

Targets prior probability shift. A multiclass distribution-matching method that replaces the per-class histograms of HDy with a single multivariate kernel density estimate over the posterior vectors on the simplex. KDEyCS minimises the Cauchy-Schwarz divergence, which has a closed form for Gaussian KDEs. Native multiclass; scales to many classes where histogram matching fragments.

Parameters

estimator	A probabilistic classifier (posterior vectors).
bandwidth	float, KDE bandwidth; controls density smoothness.
solver	Simplex optimiser / search used to minimise the objective.

Attributes

estimator_	The fitted underlying classifier.
representation_	Per-class fitted KDE models.
classes_	Class labels seen during fit.

Notes

Closed-form and the most efficient KDEy variant: the expensive train-train Gaussian integrals B are computed once at training time.

See Also

KDEyML, KDEyHD, KDEyCS, GKDEyML

Examples

```
>>> from mlquantify.matching import KDEyCS
>>> q = KDEyCS(LogisticRegression(), bandwidth=0.1).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Prior shift on the $(n-1)$ -simplex of posterior vectors. Each class i is modelled by a density p_i fitted with KDE; the test posteriors are assumed drawn from the prevalence mixture $\sum_i p_{i_prev} * p_i$. KDE

replaces histograms so the multivariate simplex is not fragmented into bins.

Algorithm / objective

Minimise the Cauchy-Schwarz divergence, available in closed form (train-train kernel matrices are pre-computed):

```
minimize_p    D_CS( sum_i p_i*KDE_i ,  KDE_test )
    D_CS(f,g) = -log(  / sqrt(  ) )
    closed form ~  r^T B r    with B precomputed at fit time
    subject to  p >= 0 ,  sum_i p_i = 1
```

Per-class KDEs are fitted once at training time; only the objective changes between the ML, HD and CS variants.

Assumptions & when to use

Assumes posteriors are shift-invariant per class and that the bandwidth is well chosen (over-smoothing flattens class differences). Excels at multiclass quantification with moderate-to-many classes; KDEyML benefits from recalibrated posteriors.

Relationship to other methods

The KDE generalisation of histogram matching (HDy/DyS). KDEyML is a likelihood method (compare EMQ); KDEyHD/KDEyCS are distribution-matching with Hellinger / Cauchy-Schwarz divergences.

References

[1] Moreo, A., González, P., & del Coz, J. J. (2024). Kernel Density Estimation for Multiclass Quantification. *arXiv:2401.00490*.

GHDy — Generalized Hellinger Distance y

Matching family | from mlquantify.matching import GHDy

API DOCSTRING · interface

Summary

Generalized Hellinger Distance y (GHDy) quantifier.

Description

Targets prior probability shift. The multiclass, constrained-regression form of HDy: it builds per-class probability-mass histograms of the posteriors and solves on the simplex for the prevalence that matches the test histogram under a Hellinger objective. Native multiclass, requires soft posteriors.

Parameters

estimator	A probabilistic classifier (posteriors).
bins	histogram bin count(s) over the posterior scores.
solver	Simplex optimiser, default 'slsqp'.

Attributes

estimator_	The fitted underlying classifier.
representation_	Per-class posterior histograms (PMF).
classes_	Class labels seen during fit.

Notes

Uses the Hellinger-surrogate loss so it can be optimised with gradient-free simplex solvers; the binary HDy is recovered for two classes.

See Also

HDy, GHDx, GACC

Examples

```
>>> from mlquantify.matching import GHdY
>>> q = GHdY(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory

Problem formulation

Firat's framework with the HDy transform: $f(x) = \text{bin}(P(.|x))$ builds a per-class posterior PMF. X is the matrix of class histograms, y the test histogram.

Algorithm / objective

Solve the constrained regression with a Hellinger objective on the simplex:

```
minimize_p    1 - sum_i sqrt( y_i * (X p)_i )
subject to    p >= 0 ,    sum_k p_k = 1
```

Equivalent to per-bin matching aggregated through the simplex solve; reduces to HDy in the binary case.

Assumptions & when to use

Posterior histograms must be shift-invariant per class and distinct across classes. Preferred over running HDy One-vs-All for genuine multiclass problems.

Relationship to other methods

Multiclass HDy. Shares the constrained-regression backbone with GACC/GPACC, differing in transform (histogram) and loss (Hellinger).

References

- [1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.
- [2] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

GHDx — Generalized Hellinger Distance x

Matching family | from mlquantify.matching import GHDx

API DOCSTRING · interface

Summary

Generalized Hellinger Distance x (GHDx) quantifier.

Description

Targets prior probability shift. The multiclass, classifier-free form of HDx: it builds per-feature histograms and solves on the simplex for the prevalence matching the test feature histograms under a Hellinger objective. Needs only raw features.

Parameters

bins	histogram bin count(s) per feature.
solver	Simplex optimiser, default 'slsqp'.

Attributes

representation_	Per-class, per-feature histograms.
classes_	Class labels seen during fit.

Notes

Classifier-free; concatenates per-feature histogram blocks and matches them jointly on the simplex.

See Also

HDx, GHDy

Examples

```
>>> from mlquantify.matching import GHDx
>>> q = GHDx().fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.34, 1: 0.33, 2: 0.33}
>>> q.aggregate(X_test)
{0: 0.34, 1: 0.33, 2: 0.33}
```

USER GUIDE · theory

Problem formulation

Firat's framework with the HDx transform $f(x)$ = per-feature histogram, matched without any classifier.

Algorithm / objective

Concatenate per-feature histogram blocks and minimise the Hellinger objective on the simplex:

```
minimize_p 1 - sum_i sqrt( y_i * (X p)_i )
subject to p >= 0 , sum_k p_k = 1
(X stacks the per-feature class histogram blocks)
```

No cross-validation; the per-feature blocks are built directly from the training features.

Assumptions & when to use

Each feature's class distribution must be shift-invariant and informative. Use when no trustworthy classifier is available; weak with many noisy features.

Relationship to other methods

Multiclass, feature-space counterpart of GHDy; the classifier-free branch of the unified framework.

References

- [1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.
- [2] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

GKDEyML — Generalized KDEy — Maximum Likelihood

Matching family | from mlquantify.matching import GKDEyML

API DOCSTRING · interface

Summary

Generalized KDEy Maximum Likelihood (GKDEyML) quantifier.

Description

Targets prior probability shift. A compose-based maximum-likelihood KDE quantifier: per-class kernel densities feed a mixture-likelihood objective solved on the simplex. The generalised, pluggable-representation sibling of KDEyML.

Parameters

estimator	A probabilistic classifier (posteriors).
bandwidth	float, KDE bandwidth.
solver	Simplex optimiser used to maximise the likelihood.

Attributes

estimator_	The fitted underlying classifier.
representation_	Per-class fitted KDE models.
classes_	Class labels seen during fit.

Notes

Shares KDEyML's objective but is wired through the compose / likelihood machinery, so the representation and loss can be swapped.

See Also

KDEyML, KDEyHD, KDEyCS

Examples

```
>>> from mlquantify.matching import GKDEyML
>>> q = GKDEyML(LogisticRegression(), bandwidth=0.1).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.25, 1: 0.40, 2: 0.35}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.25, 1: 0.40, 2: 0.35}
```

USER GUIDE · theory

Problem formulation

Same simplex / posterior-density setting as KDEyML, expressed through the likelihood-compose interface.

Algorithm / objective

Maximise the mixture log-likelihood of the test posteriors under the per-class KDEs:

$$\begin{aligned} &\text{maximize}_p \quad \sum_{x \text{ in test}} \log(\sum_i p_i * \text{KDE}_i(x)) \\ &\text{subject to} \quad p \geq 0, \quad \sum_i p_i = 1 \end{aligned}$$

Identical objective to KDEyML; the compose form lets the density representation be replaced.

Assumptions & when to use

As KDEyML: shift-invariant posteriors and a sensible bandwidth; benefits from calibrated probabilities. For multiclass likelihood-based quantification.

Relationship to other methods

The compose-framework rendering of KDEyML, paired with the likelihood losses used by EMQ and MLPE.

References

[1] Moreo, A., González, P., & del Coz, J. J. (2024). Kernel Density Estimation for Multiclass Quantification. *arXiv:2401.00490*.

EDy — Energy Distance y

Matching family | from mlquantify.matching import EDy

API DOCSTRING · interface

Summary

Energy Distance y (EDy) quantifier.

Description

Targets prior probability shift. A distribution-matching quantifier that represents each class by the full set of its classifier predictions and minimises the energy distance between the test set and the prevalence-weighted mixture of class sets. Native multiclass and strong in the multiclass regime.

Parameters

estimator	A probabilistic classifier (predictions).
metric	Ground distance delta between predictions used in the energy term; default Manhattan. - manhattan — L1 distance between prediction vectors; the paper's default for EDy. - euclidean — L2 distance between prediction vectors. - cityblock — alias of Manhattan as accepted by scipy cdist.

solver	Simplex optimiser, default 'slsqp'.
Attributes	
estimator_	The fitted underlying classifier.
representation_	Per-class distance representation.
classes_	Class labels seen during fit.

Notes

The energy objective is a quadratic form $p^T(2q-Mp)$; distributions are estimated via cross-validation for both train and test. EDy uses classifier predictions, EDx uses raw features.

See Also

EDx, MMD_RKHS

Examples

```
>>> from mlquantify.matching import EDy
>>> q = EDy(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.2, 1: 0.5, 2: 0.3}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.2, 1: 0.5, 2: 0.3}
```

USER GUIDE · theory**Problem formulation**

Prior shift matched with the energy distance. Model the test set T as the mixture $D' = \sum_l p_l * D_{cl}$ of the per-class prediction sets; minimise the energy distance $ED(T, D')$ over the prevalence p , with δ a ground distance between predictions.

Algorithm / objective

Dropping the test-only term, the energy distance becomes a quadratic form in the prevalence:

```
minimize_p    2 * sum_l p_l * E[ delta(x_l, x_test) ]
              - sum_l sum_l' p_l p_l' * E[ delta(x_l, x_l') ]
==  p^T ( 2 q - M p )      (mlquantify EnergyLoss)
q_l  = mean delta(class l, test)
M_ll' = mean delta(class l, class l')
subject to    p >= 0 ,    sum_l p_l = 1
```

EDy sets δ on classifier predictions $\delta(h(x_i), h(x_j))$; EDx sets it on raw features. Solved on the simplex.

Assumptions & when to use

Assumes prediction (or feature) distributions are shift-invariant per class. Particularly effective for multiclass tasks; cost grows with the number of cross distances. Calibrated posteriors help EDy.

Relationship to other methods

Energy-distance sibling of MMD_RKHS (kernel norm vs energy distance). EDx is the feature-space version using the original Kawakubo energy-distance estimator.

References

- [1] del Coz, J. J. (2022). UniOviedo(Team2) at LeQua 2022: Comparison of Traditional Quantifiers and a New Method Based on Energy Distance. *CLEF 2022*.
- [2] Kawakubo, H., du Plessis, M. C., & Sugiyama, M. (2016). Computationally Efficient Class-Prior Estimation under Class Balance Change Using Energy Distance. *IEICE Trans.*, 99(1), 176–186.

EDx — Energy Distance x

Matching family | from mlquantify.matching import EDx

API DOCSTRING · interface

Summary

Energy Distance x (EDx) quantifier.

Description

Targets prior probability shift. The classifier-free energy-distance quantifier: it computes the energy distance directly on raw feature vectors between the test set and the prevalence mixture of per-class sets. Multiclass; needs no scorer.

Parameters

metric	Ground distance on features, default Euclidean/Manhattan.
solver	Simplex optimiser, default 'slsqp'.

Attributes

representation_	Per-class distance representation in feature space.
classes_	Class labels seen during fit.

Notes

Energy distance on raw features (Kawakubo et al.); avoids classifier calibration but ignores any learned representation.

See Also

EDy, MMD_RKHS

Examples

```
>>> from mlquantify.matching import EDx
>>> q = EDx().fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.34, 1: 0.33, 2: 0.33}
>>> q.aggregate(X_test)
{0: 0.34, 1: 0.33, 2: 0.33}
```

USER GUIDE · theory

Problem formulation

Same energy-distance matching as EDy, but the ground distance delta acts on raw feature vectors, so no classifier is needed.

Algorithm / objective

Minimise the same energy quadratic form with feature-space distances:

```
minimize_p    p^T ( 2 q - M p )
    q_l  = mean delta_features(class l, test)
    M_ll' = mean delta_features(class l, class l')
subject to    p >= 0 ,  sum_l p_l = 1
```

Distances are computed in input space; estimator-free.

Assumptions & when to use

Feature distributions must be shift-invariant per class and meaningfully metric. Use when a classifier is unavailable or untrusted; sensitive to feature scaling.

Relationship to other methods

Feature-space version of EDy and the original energy-distance estimator that EDy adapts to classifier predictions.

References

- [1] Kawakubo, H., du Plessis, M. C., & Sugiyama, M. (2016). Computationally Efficient Class-Prior Estimation under Class Balance Change Using Energy Distance. *IEICE Trans.*, 99(1), 176–186.
 - [2] del Coz, J. J. (2022). UniOviedo(Team2) at LeQua 2022: Comparison of Traditional Quantifiers and a New Method Based on Energy Distance. *CLEF 2022*.
-

Likelihood

Likelihood methods estimate prevalence by fitting the test priors that best explain the observed test data — either by maximising a mixture likelihood or by iteratively re-estimating the class distribution. They adjust a fixed classifier's outputs without retraining it.

EMQ — Expectation-Maximisation Quantifier

Likelihood family | `from mlquantify.likelihood import EMQ`

API DOCSTRING · interface

Summary

Expectation-Maximisation Quantifier (EMQ) quantifier.

Description

Targets prior probability shift. Runs the Saerens-Latinne-Decaestecker EM procedure: it alternately rescales the classifier posteriors by the current prior estimate and re-averages them, converging to the test priors that maximise the test-set likelihood. Requires soft, well-calibrated posteriors; no classifier retraining.

Parameters

estimator	A probabilistic classifier with <code>predict_proba</code> .
max_iter	int, maximum EM iterations.
tol	float, convergence threshold on the prevalence change between iterations.

Attributes

estimator_	The fitted underlying classifier.
priors_	Training class prevalences used to initialise EM.
classes_	Class labels seen during fit.

Notes

EMQ (a.k.a. SLD) is highly sensitive to calibration: bias-corrected temperature scaling of the posteriors before EM (Alexandari 2020) makes it hard to beat. It corrects classifier bias without retraining.

See Also

CDE, MLPE

Examples

```
>>> from mlquantify.likelihood import EMQ
>>> q = EMQ(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.31, 1: 0.69}
>>> proba = q.estimator_.predict_proba(X_test)
>>> q.aggregate(proba)
{0: 0.31, 1: 0.69}
```

USER GUIDE · theory

Problem formulation

Prior shift where only the priors $p(y)$ change. Given a classifier trained with priors $p_{tr}(y)$ and its posteriors $P(y|x)$, EM seeks new priors $p_{te}(y)$ that maximise the likelihood of the unlabelled test bag under the corrected posteriors.

Algorithm / objective

Initialise the priors to the training priors and iterate the E and M steps until convergence:

- 1. E-step** — rescale posteriors by the current prior ratio and renormalise.
- 2. M-step** — set the new prior of each class to the mean adjusted posterior.

3. Repeat until the prior estimate stops changing (tol) or max_iter is reached.

```
E:  P^(s)(y|x) = [ (p^(s)(y)/p_tr(y)) * P(y|x) ]
                / sum_k (p^(s)(k)/p_tr(k)) * P(k|x)

M:  p^(s+1)(y) = (1/n) * sum_{x in test} P^(s)(y|x)
```

Converges to a local maximum of the test log-likelihood, usually within a few iterations; the final priors are the prevalence estimate.

Assumptions & when to use

Unbiased under prior shift provided the posteriors are calibrated on the test distribution; mis-calibration is its main failure mode. One of the strongest quantifiers when paired with calibration.

Relationship to other methods

Likelihood counterpart of the distribution-matching family. KDEyML maximises a similar mixture likelihood with KDE-smoothed densities; MLPE is the no-iteration baseline.

References

- [1] Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the Outputs of a Classifier to New a Priori Probabilities: A Simple Procedure. *Neural Computation*, 14(1), 21–41.
- [2] Alexandari, A., Kundaje, A., & Shrikumar, A. (2020). Maximum Likelihood with Bias-Corrected Calibration is Hard-to-Beat at Label Shift Adaptation. *ICML*.

CDE — CDE-Iterate

Likelihood family | from mlquantify.likelihood import CDE

API DOCSTRING · interface

Summary

Class Distribution Estimation by iteration (CDE) quantifier.

Description

Targets prior probability shift. Iteratively re-estimates the test class distribution and rebuilds the classifier with a cost ratio reflecting the current estimate, shifting the effective decision threshold toward the test distribution at each pass. Binary base method.

Parameters

estimator	A cost-sensitive base classifier.
max_iter	int, number of CDE iterations.
tol	float, convergence threshold on the estimate.

Attributes

estimator_	The classifier from the last iteration.
priors_	Training class prevalences.
classes_	Class labels seen during fit.

Notes

Each iteration adjusts the FP/FN cost ratio from the estimated Pos:Neg ratio, so the decision boundary tracks the test distribution. Usually converges in a few iterations.

See Also

EMQ, MLPE

Examples

```
>>> from mlquantify.likelihood import CDE
>>> q = CDE(LogisticRegression()).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.40, 1: 0.60}
>>> preds = q.estimator_.predict(X_test)
>>> q.aggregate(preds, y_train=y_train)
{0: 0.40, 1: 0.60}
```

USER GUIDE · theory

Problem formulation

Binary prior shift where the changed class distribution biases a fixed classifier. CDE estimates the new distribution and feeds it back as a cost ratio to re-bias the model.

Algorithm / objective

Iteratively classify, estimate, and re-weight:

1. Build the initial classifier on the original labelled data and classify the test set.
2. Estimate the new class distribution (Pos:Neg) from those predictions.
3. Rebuild / re-threshold the classifier with a cost ratio derived from the estimate.
4. Repeat for a few iterations; return the final distribution.

```
cost_ratio <- Pos2Neg(original) adjusted by current NEW estimate
C_i        <- build_classifier( data , cost_ratio )
p_hat      <- distribution of C_i on the test set
```

Each pass moves the effective threshold toward the test distribution, refining the prevalence.

Assumptions & when to use

Assumes prior shift and a cost-sensitive learner. Can oscillate if the classifier is unstable; best when the base learner responds smoothly to cost weighting. Binary.

Relationship to other methods

Like EMQ it iteratively reconciles predictions with an evolving prior, but it re-biases the classifier (threshold/costs) rather than rescaling posteriors analytically.

References

- [1] Xue, J. C., & Weiss, G. M. (2009). Quantification and Semi-Supervised Classification Methods for Handling Changes in Class Distribution. *KDD*.

MLPE — Maximum Likelihood Prevalence Estimation

Likelihood family | from mlquantify.likelihood import MLPE

API DOCSTRING · interface

Summary

Maximum Likelihood Prevalence Estimation (MLPE) quantifier.

Description

Assumes no shift. Predicts the training class prevalence for every test bag — the maximum-likelihood estimate when the test prior is assumed equal to the training prior. Ignores the test features entirely; serves as the trivial lower-bound baseline.

Parameters

estimator Optional; unused for prediction (kept for a uniform interface).

Attributes

priors_ Training class prevalences returned as the estimate.

classes_ Class labels seen during fit.

Notes

MLPE is the 'no-change' reference: any useful quantifier should beat it under real shift. Its error equals the shift magnitude.

See Also

EMQ, CDE

Examples

```
>>> from mlquantify.likelihood import MLPE
>>> q = MLPE().fit(X_train, y_train)
>>> q.predict(X_test)          # always the training prevalence
{0: 0.50, 1: 0.50}
>>> q.aggregate(y_train=y_train)
{0: 0.50, 1: 0.50}
```

USER GUIDE · theory

Problem formulation

Degenerate case of prior estimation: assume $p_{te}(y) = p_{tr}(y)$. The multinomial likelihood of the (unobserved) test labels is then maximised by the training frequencies.

Algorithm / objective

Return the training prevalence regardless of the test bag:

$$p_{\text{hat}}(y) = p_{\text{tr}}(y) = (1/n_{\text{tr}}) * \sum_{x \text{ in train}} I(y_x = y)$$

No optimisation and no use of the test data.

Assumptions & when to use

Correct only when there is no shift. Useful purely as a baseline and sanity check; never use it when the test distribution is expected to move.

Relationship to other methods

The null model of the likelihood family — EMQ and CDE reduce to it when the test bag carries no evidence of shift.

References

- [1] González, P., Castaño, A., Chawla, N. V., & del Coz, J. J. (2017). A Review on Quantification Learning. *ACM Computing Surveys*, 50(5), 74.

Neighbors

Neighbour methods estimate prevalence from a nearest-neighbour vote, reweighting the votes to undo the majority-class bias that ordinary k-NN inherits under class imbalance.

PWK — Proportion-Weighted k-NN

Neighbors family | from mlquantify.neighbors import PWK

API DOCSTRING · interface

Summary

Proportion-Weighted k-Nearest-Neighbour (PWK) quantifier.

Description

Targets prior probability shift. Classifies test instances with a k-NN whose neighbour votes are reweighted to compensate the bias toward the majority class, then counts the weighted predictions. The weighting (PWKa, exponent a) interpolates between plain k-NN and full prevalence-compensation. Binary base method.

Parameters

n_neighbors	int, number of neighbours k.
alpha	float, weighting exponent a (a=1 = PWK; large a -> plain k-NN).
metric	Distance metric for the neighbour search.

Attributes

classifier_	The fitted weighted k-NN classifier.
classes_	Class labels seen during fit.

Notes

Weights are inversely related to each class's training prevalence, so the minority class is not swamped; PWK is among the few NN quantifiers to beat CC/ACC significantly.

See Also

CC, ACC

Examples

```
>>> from mlquantify.neighbors import PWK
>>> q = PWK(n_neighbors=10).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.45, 1: 0.55}
>>> preds = q.classifier_.predict(X_test)
>>> q.aggregate(preds, y_train=y_train)
{0: 0.45, 1: 0.55}
```

USER GUIDE · theory

Problem formulation

Binary prior shift. Ordinary k-NN counts are biased toward the majority class because dense regions dominate the neighbour sets. PWK rebalances the vote before counting.

Algorithm / objective

Weight each neighbour's vote to offset its class's training prevalence, classify, then classify-and-count:

$$w(\text{class } c) \sim (\text{prior_tr}(c))^{-1/a} \quad (\text{down-weight majority class})$$

$$\text{label}(x) = \underset{c}{\operatorname{argmax}} \sum_{j \in \text{kNN}(x)} w(\text{class of } j) \cdot I(\text{class } j = c)$$

$$\hat{p}(c) = (1/n) \cdot \sum_{x \in \text{test}} I(\text{label}(x) = c)$$

a=1 recovers PWK; as a grows the weights flatten to 1 and PWK -> standard k-NN.

Assumptions & when to use

Assumes prior shift and a meaningful distance metric. Robust and simple; like all k-NN it suffers in high dimensions and needs feature scaling. Binary focus.

Relationship to other methods

A counting method whose base predictor is a bias-corrected k-NN rather than a thresholded scorer; PWKa generalises it with KNN and PWK as the extremes of a.

References

- [1] Barranquero, J., González, P., Díez, J., & del Coz, J. J. (2013). On the Study of Nearest Neighbor Algorithms for Prevalence Estimation in Binary Problems. *Pattern Recognition*, 46(2), 472–482.
-

Meta

Meta quantifiers wrap a base quantifier to add a higher-level capability: ensemble diversity and selection, bootstrap-based uncertainty, or on-the-fly adaptation to distribution shift. They compose with any of the methods above.

EnsembleQ — Ensemble Quantifier

Meta family | from mlquantify.meta import EnsembleQ

API DOCSTRING · interface

Summary

Ensemble Quantifier with prevalence-controlled diversity (EnsembleQ).

Description

Targets prior probability shift (and characterisable distribution change). Trains many copies of a base quantifier on subsamples drawn at deliberately different class prevalences, then aggregates their estimates — optionally selecting only the members whose training distribution resembles the test sample (dynamic selection). Reduces variance and adapts to the test shift.

Parameters

quantifier	The base quantifier replicated across ensemble members.
size	int, default=50. Number of ensemble members.
min_prop, max_prop	Prevalence range used when sampling training batches; sets the diversity.
selection_metric	Which members vote at prediction time. <code>all</code> — use every member (plain bagged average; no selection). <code>ptr</code> — keep members whose training prevalence is closest to an initial test estimate. <code>ds</code> — keep members whose training score distribution is closest to the test distribution.
p_metric	Fraction of members retained when a selection metric (<code>ptr/ds</code>) is used; smaller keeps only the most relevant members.
protocol	Protocol used to draw each member's training prevalence. <code>uniform</code> — prevalences spread uniformly over the simplex (default). <code>artificial</code> — fixed grid of prevalences (artificial-prevalence protocol). <code>natural</code> — prevalences resampled from the natural training distribution. <code>kraemer</code> — Kraemer sampling of prevalence vectors on the simplex.
return_type	Aggregation over the selected members. <code>mean</code> — average of the member estimates (default). <code>median</code> — median of the member estimates; more robust to outlier members.

Attributes

models	Fitted ensemble member quantifiers.
train_prevalences	Training prevalence of each member.
classes	Class labels seen during fit.

Notes

Members are trained with sampling-with-replacement so that $p(x|y)$ is preserved while only $p(y)$ varies. Dynamic selection ('ptr'/ds') is what lets the ensemble specialise to the test shift; with 'all' it is a plain bagged average.

See Also

AggregativeBootstrap, QuaDapt

Examples

```
>>> from mlquantify.meta import EnsembleQ
>>> from mlquantify.matching import DyS
>>> q = EnsembleQ(DyS(LogisticRegression()), size=50).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.49, 1: 0.51}
```

USER GUIDE · theory

Problem formulation

Prior shift whose magnitude is unknown at training time. A single quantifier is tuned to one training prevalence; an ensemble spanning many prevalences brackets the possible test distributions and lets the relevant members dominate.

Algorithm / objective

Build a prevalence-diverse ensemble, then aggregate (optionally select):

1. For m members, draw a training sample at a target prevalence (sampling with replacement, preserving $p(x|y)$); train a base quantifier on each.
2. At test time, optionally select the members whose training distribution is most similar to the test sample (ptr: prevalence similarity; ds: score-distribution similarity).
3. Aggregate the selected members' prevalence estimates by mean or median.

```
members: q_1, ..., q_m trained at prevalences pi_1, ..., pi_m
select:   S = top-p_metric members by similarity( pi_j , test )
p_hat = aggregate_{j in S} q_j(test)
```

Dynamic selection adapts the ensemble to each test bag without retraining.

Assumptions & when to use

Assumes the test shift is 'characterisable' — covered by the range of training prevalences sampled. Most useful when the shift is large or unknown; cost scales with the ensemble size. Pair with a strong base quantifier (e.g. DyS, EMQ).

Relationship to other methods

A wrapper, not a new estimator: it amplifies any base method (counting, matching or likelihood) with diversity and dynamic selection.

References

- [1] Pérez-Gállego, P., Quevedo, J. R., & del Coz, J. J. (2017). Using Ensembles for Problems with Characterizable Changes in Data Distribution. *Information Fusion*, 34, 87–100.
- [2] Pérez-Gállego, P., Castaño, A., Quevedo, J. R., & del Coz, J. J. (2019). Dynamic Ensemble Selection for Quantification Tasks. *Information Fusion*, 45, 1–15.

AggregativeBootstrap — Aggregative Bootstrap

Meta family | `from mlquantify.meta import AggregativeBootstrap`

API DOCSTRING · interface

Summary

Aggregative Bootstrap quantifier for prevalence confidence regions.

Description

Targets prior probability shift. Wraps an aggregative quantifier and bootstrap-resamples its predictions to turn a point prevalence estimate into a confidence region. Because aggregative quantifiers classify once and aggregate, the resampling is applied to the cheap aggregation step, giving uncertainty estimates efficiently.

Parameters

quantifier The base aggregative quantifier to wrap.

n_train_bootstraps	int, resamples of the training predictions.
n_test_bootstraps	int, resamples of the test predictions.
region_type	Shape of the confidence region built from the resampled estimates. <code>intervals</code> — independent per-class percentile intervals (default; simplest to read). <code>ellipse</code> — Gaussian confidence ellipse in the simplex; captures class correlations. <code>ellipse-clr</code> — ellipse in centered-log-ratio (Aitchison) space; respects simplex geometry.
confidence_level	float, default=0.95. Probability mass the region is meant to cover.
Attributes	
train_predictions	Cached predictions on the training/validation set.
y_train	Labels for the cached predictions.
classes	Class labels seen during fit.

Notes

Combining train- and test-side resampling captures both model and sampling uncertainty. The CLR ellipse lives in Aitchison (log-ratio) space, respecting the simplex geometry.

See Also

EnsembleQ, ConfidenceInterval

Examples

```
>>> from mlquantify.meta import AggregativeBootstrap
>>> from mlquantify.likelihood import EMQ
>>> q = AggregativeBootstrap(EMQ(LogisticRegression()),
...                           n_train_bootstraps=10, n_test_bootstraps=10).fit(X_train, y_train)
>>> q.predict(X_test)      # point estimate (+ stored region)
{0: 0.31, 1: 0.69}
```

USER GUIDE · theory

Problem formulation

A point prevalence estimate gives no sense of its reliability. Bootstrap resampling approximates the sampling distribution of the estimate, from which a confidence region at a chosen level is read off.

Algorithm / objective

Exploit the two-phase structure of aggregative quantifiers:

1. Run the expensive classification phase once, caching the train and test predictions.
2. Resample the cached predictions with replacement (train and/or test sides) and re-run only the aggregation, producing many prevalence estimates.
3. Summarise the resampled estimates as a point estimate plus a confidence region (per-class interval, simplex ellipse, or CLR-space ellipse).

```
for b = 1..B: p_hat^(b) = aggregate( resample(predictions) )
region = level-set of { p_hat^(1), ..., p_hat^(B) } at confidence_level
```

Applying bootstrap only to aggregation gives large speed-ups versus resampling the whole pipeline.

Assumptions & when to use

Inherits the base quantifier's prior-shift assumption; the region is valid in so far as the bootstrap approximates the true sampling distribution (needs enough test points and resamples). Use whenever a decision needs an uncertainty band, not just a number.

Relationship to other methods

A wrapper that adds uncertainty quantification to any aggregative method; it produces the confidence regions defined in the confidence module.

References

[1] Moreo, A., & Salvati, M. (2025). An Efficient Method for Deriving Confidence Intervals in Aggregative Quantification. *LQ 2025 / arXiv*.

QuaDapt — Drift-Resilient Quantification via Parameters Adaptation

Meta family | from `mlquantify.meta` import `QuaDapt`

API DOCSTRING · interface

Summary

QuaDapt: adaptive quantification via synthetic score simulation.

Description

Targets general distribution shift / concept drift, not only prior shift. Wraps a soft base quantifier and, at prediction time, simulates training score distributions with MoSS at several overlap levels, selecting the one whose mixed scores best match the test scores; that synthetic set becomes the aggregation reference. Binary base method (One-vs-Rest for multiclass).

Parameters

quantifier	A soft (probabilistic) base aggregative quantifier.
measure	Distance comparing each synthetic score set with the test scores, used to pick the best merging factor; default <code>'topsoe'</code> . <code>topsoe</code> — symmetric information-theoretic distance (default). <code>hellinger</code> — bounded sqrt-probability distance. <code>probsymm</code> — probabilistic symmetric chi-square distance. <code>sord</code> — Sample Ordinal Distance on the raw scores (bin-free).
merging_factors	Candidate MoSS overlap levels <code>m</code> to evaluate; default 0.1–0.9 in steps of 0.2. Each <code>m</code> sets how much the synthetic positive and negative scores overlap (0 = well separated, 1 = fully overlapping).
strategy	Multiclass decomposition into binary sub-problems. <code>ovr</code> — one-vs-rest: one binary adaptation per class (default). <code>ovo</code> — one-vs-one: one binary adaptation per class pair.

Attributes

classes_	Class labels seen during fit.
y_train	Training labels stored at fit.

Notes

Built on MoSS (*Model for Score Simulation*): the merging factor `m` sets class-score overlap (0 = well separated, 1 = heavy overlap). Adapting `m` to the test scores makes a standard quantifier resilient when the score-distribution complexity drifts.

See Also

DyS, EnsembleQ

Examples

```
>>> from mlquantify.meta import QuaDapt
>>> from mlquantify.matching import DyS
>>> q = QuaDapt(DyS(LogisticRegression())).fit(X_train, y_train)
>>> q.predict(X_test)
{0: 0.49, 1: 0.51}
>>> proba = LogisticRegression().fit(X_train, y_train).predict_proba(X_test)
>>> q.aggregate(proba, y_train)
{0: 0.49, 1: 0.51}
```

USER GUIDE · theory

Problem formulation

Beyond prior shift, the very complexity (overlap) of the class score distributions can drift between train and test, breaking quantifiers that assume fixed class-conditional scores. QuaDapt makes no assumption on the drift form; it realigns the training reference to the observed test scores.

Algorithm / objective

MoSS generates synthetic positive/negative scores parameterised by a merging factor m (overlap) and a positive proportion α :

1. For each candidate merging factor m , build MoSS synthetic positive/negative scores.
2. Pick the m whose mixed synthetic distribution is closest to the test scores under the chosen distance (topsoe by default).
3. Use that synthetic, drift-aligned score set as the training reference for the base quantifier's aggregate, and estimate the prevalence.

```
MoSS(n, alpha, m) = syn(+, floor(alpha*n), m) U syn(-, floor((1-alpha)*n), m)
syn(+, n, m) = {  $X_i^m$  },  $X_i \sim U(0,1)$ 
syn(-, n, m) = {  $1 - X_i^m$  },  $X_i \sim U(0,1)$ 
```

Only the quantifier's internal score reference is adapted — no classifier retraining. Generalises DySyn (DyS + MoSS) to any classifier-based quantifier.

Assumptions & when to use

Does not assume a specific drift type; assumes scores can be realigned by a one-parameter overlap model (MoSS). Most valuable under concept/covariate drift that changes score complexity, where prior-shift quantifiers fail. Binary core, OvR for multiclass.

Relationship to other methods

A drift-resilient wrapper for any soft quantifier; extends Maletzke et al.'s MoSS-based DySyn from DyS to the whole quantifier family.

References

- [1] Ortega, J. P., Luth Junior, L. F., Zalewski, W., & Maletzke, A. (2025). QuaDapt: Drift-Resilient Quantification via Parameters Adaptation. *Proc. 5th Int. Workshop on Learning to Quantify (LQ 2025)*, p. 64.
- [2] Maletzke, A., dos Reis, D., Hassan, W., & Batista, G. (2021). Accurately Quantifying under Score Variability. *ICDM 2021*, 1228–1233. (Model for Score Simulation, MoSS.)

PART II

Solvers

Optimisation backends that convert a matching or likelihood objective into a prevalence vector over $[0,1]$ or the probability simplex.

Solvers

Solvers are the optimisation backends shared by the quantifier families. They turn a distribution-matching or likelihood objective into a prevalence vector, either over the binary interval [0,1] or over the full probability simplex.

solve_binary — Binary prevalence solver

`solver` | from mlquantify.solvers import solve_binary

API · interface

Summary

Minimise a scalar objective over the binary prevalence space [0, 1].

Description

Optimises a one-dimensional objective $f(\alpha)$, where α is the positive-class prevalence, by exhaustive grid search, ternary search (unimodal objectives), or scipy's bounded scalar minimiser. Returns the two-class prevalence vector and the objective value.

Signature & parameters

objective	callable $f(\alpha) \rightarrow \text{float}$, α in [0,1].
solver	Optimisation strategy over α . • <code>auto</code> — selects 'bounded' (default). • <code>grid</code> — evaluate the objective at <code>grid_size</code> evenly-spaced points; robust, no unimodality needed. • <code>ternary</code> — interval-trisection search; fastest when the objective is unimodal. • <code>bounded</code> — scipy bounded scalar minimiser.
grid_size	int, number of candidate α values for the grid solver.
tol	float, convergence tolerance for the ternary solver.

Returns

prevalence	ndarray (2,) = [1 - α , α].
loss	float, objective value at the optimum.

Example

```
>>> from mlquantify.solvers import solve_binary
>>> p, loss = solve_binary(lambda a: (a - 0.3)**2, solver='bounded')
>>> round(p[1], 2)
0.3
```

USER GUIDE · role & theory

Role & mechanism

The backbone of the binary matching methods. The dissimilarity between the α -mixed class representation and the test representation is unimodal in α for most distances, so ternary or bounded search converges quickly and robustly.

```
alpha* = argmin_{0 <= alpha <= 1} objective(alpha)
return [ 1 - alpha* , alpha* ]
```

Used by

DyS, HDy, HDx, SORD, SMM and any binary distribution-matching quantifier.

References

[1] Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.

ternary_search — Ternary search

solver | from mlquantify.solvers import ternary_search

API · interface

Summary

Find the minimum of a unimodal function by interval trisection.

Description

Repeatedly evaluates the objective at two interior probes and discards the third of the interval that cannot contain the minimum, until the interval is narrower than tol.

Signature & parameters

left, right	float, bounds of the search interval.
objective	callable, unimodal on [left, right].
tol	float, stop when $\text{right} - \text{left} \leq \text{tol}$.

Returns

minimum	float, approximate location of the minimum.
----------------	---

Example

```
>>> from mlquantify.solvers import ternary_search
>>> round(ternary_search(0.0, 1.0, lambda x: (x-0.3)**2), 4)
0.3
```

USER GUIDE · role & theory

Role & mechanism

A derivative-free line search that exploits unimodality: each iteration shrinks the interval by one third, giving linear convergence without gradients.

```
m1 = left + (right-left)/3 ; m2 = right - (right-left)/3
if f(m1) < f(m2): right = m2 else: left = m1 # repeat until width <= tol
```

Used by

solve_binary (the 'ternary' strategy) and, through it, the DyS family.

solve_simplex — Simplex (SLSQP) solver

solver | from mlquantify.solvers import solve_simplex

API · interface

Summary

Minimise a function over the probability simplex using SLSQP.

Description

Minimises a multivariate objective $f(p)$ subject to the simplex constraints ($p \geq 0$, $\sum p = 1$) via sequential least-squares programming; the result is clipped to non-negative values and renormalised.

Signature & parameters

objective	callable $f(p) \rightarrow \text{float}$, p of shape $(n_classes,)$.
n_classes	int, dimensionality of p .
x0	optional initial guess (defaults to uniform).
random_state	seed for a random simplex start.

Returns

prevalence	ndarray $(n_classes,)$ summing to 1.
-------------------	---------------------------------------

loss float, objective value at the optimum.

Example

```
>>> from mlquantify.solvers import solve_simplex
>>> target = np.array([0.2, 0.5, 0.3])
>>> p, _ = solve_simplex(lambda p: np.sum((p-target)**2), n_classes=3)
>>> np.round(p, 2)
array([0.2, 0.5, 0.3])
```

USER GUIDE · role & theory

Role & mechanism

The multiclass workhorse: it enforces the constrained-regression feasibility set of the unified framework, returning a valid distribution for any smooth objective.

```
minimize_p objective(p)
subject to sum_k p_k = 1 , 0 <= p_k <= 1 (SLSQP)
```

Used by

GACC, GPACC, FM, GHDy, GHDx, GKDEyML, MMD_RKHS, EDy, EDx, KDEy (multiclass).

References

[1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.

minimize_prevalence — Prevalence minimisation dispatcher

solver | from mlquantify.solvers import minimize_prevalence

API · interface

Summary

Minimise an objective over the simplex, dispatching binary vs multiclass.

Description

Routes to the binary solver when `n_classes == 2` and the requested strategy is compatible, and to the SLSQP simplex solver otherwise. With `solver='auto'` it picks bounded search for binary problems and SLSQP for multiclass.

Signature & parameters

objective	callable; scalar for binary, vector for multiclass.
n_classes	int ≥ 2.
solver	Backend selection; binary strategies dispatch to <code>solve_binary</code> , 'slsqp' to <code>solve_simplex</code> . • <code>auto</code> — bounded for binary, SLSQP for multiclass (default). • <code>grid</code> — binary grid search (<code>n_classes = 2</code> only). • <code>ternary</code> — binary ternary search (<code>n_classes = 2</code> only). • <code>bounded</code> — binary bounded scalar minimiser (<code>n_classes = 2</code> only). • <code>slsqp</code> — constrained SLSQP over the full simplex (any <code>n_classes</code>).
grid_size, tol, x0, random_state	forwarded to the chosen backend.

Returns

prevalence	ndarray (<code>n_classes</code> ,) summing to 1.
loss	float, objective value at the optimum.

Example

```
>>> from mlquantify.solvers import minimize_prevalence
>>> p, _ = minimize_prevalence(lambda a: (a-0.3)**2, n_classes=2, solver='bounded')
>>> round(p[1], 2)
0.3
```

USER GUIDE · role & theory

Role & mechanism

A single entry point so a quantifier can be written once and run in both binary and multiclass mode; it hides the binary/simplex distinction behind one call.

```
n_classes == 2 and solver in {grid,ternary,bounded} -> solve_binary
otherwise (slsqp) -> solve_simplex
```

Used by

Every optimisation-based quantifier that must support both binary and multiclass.

minimize_prevalence_blocks — Block-wise prevalence minimisation

solver | from mlquantify.solvers import minimize_prevalence_blocks

API · interface

Summary

Minimise a loss per representation block, then aggregate the estimates.

Description

For each sub-vector (block) of the test and training representations — e.g. each histogram bin group — builds a block-specific objective, minimises it independently, and aggregates the per-block prevalence estimates by median or mean.

Signature & parameters

objective_factor y	callable producing a block objective from (test_block, train_block).
test/train_representations	the full representations to split into blocks.
block_slices	slice objects identifying each block.
aggregate	'median' (default) or 'mean'.

Returns

prevalence	ndarray (n_classes,), the aggregated estimate.
-------------------	--

Example

```
>>> from mlquantify.solvers import minimize_prevalence_blocks
>>> # one estimate per histogram-bin block, aggregated by the median
>>> p = minimize_prevalence_blocks(factory, q_test, q_train,
...                               block_slices, n_classes=2)
```

USER GUIDE · role & theory

Role & mechanism

Implements the 'sweep and take the median' recipe of the histogram-matching family: each bin configuration yields one estimate and the median is robust to the unreliable ones.

```
for each block b: p_b = argmin objective_b( test_b , train_b )
p_hat = median_b ( p_b )      (or mean)
```

Used by

HDy, HDx and the histogram distribution-matching quantifiers.

References

- [1] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.
-

PART III

Representations

Feature/score descriptors that a distribution-matching quantifier compares; the representation choice is what specialises a matching skeleton into HDy, EDy, MMD or KDEy.

Representations

Representations turn a sample of instances (or scores) into the fixed-length descriptor that a distribution-matching quantifier compares. Swapping the representation is what turns one matching skeleton into HDy, EDy, MMD or KDEy.

HistogramRepresentation — Histogram representation

`representation` | from `mlquantify.representations` import `HistogramRepresentation`

API · interface

Summary

Per-feature histogram representation with optional block partitioning.

Description

Bins each feature (or score) independently and concatenates the normalised bin frequencies into one vector. Supports fixed or quantile edges, one-hot mode, Laplace smoothing, and block slices for per-feature recovery.

Signature & parameters

bins	int or array of ints — how many bins each feature/score is split into. More bins capture finer distribution detail but need more samples to fill reliably; passing several values lets a quantifier sweep resolutions and take a median. This is the resolution knob of the representation.
range	(low, high) — the value interval the bins span, default (0.0, 1.0) since classifier posteriors live in [0,1]. Values outside the range fall into the edge bins. Set it to the actual feature range when binning raw features.
mode	How each value is turned into bin mass. <code>histogram</code> — count values per bin, then normalise to a probability mass function (default). <code>onehot</code> — assign each value to its bin as a one-hot vector and average; a softer per-sample encoding.
features	indices of the features to histogram; <code>None</code> uses all. Lets you restrict matching to informative features (e.g. only the positive-class posterior in the binary case).
bin_edges	How the bin boundaries are placed — the key accuracy/robustness trade-off of the histogram. <code>fixed</code> — equal-width edges from <code>range</code>; fast, no fitting, but wastes bins where data is sparse (default). <code>auto</code> — data-driven (quantile) edges learned at fit time so each bin holds a similar mass; more robust to skewed scores at the cost of storing the edges.
partition_blocks	if <code>True</code> , keep each feature's bins as a separate contiguous block and expose <code>block_slices_</code> , so a quantifier can match (and aggregate) one feature-block at a time instead of one big concatenated vector. This is what enables the per-bin median sweep of HDy/HDx.
laplace_smoothing	if <code>True</code> , add a small (1/k) count to every bin before normalising. This removes zero bins, which otherwise make ratio/log-based distances (Hellinger, Topsoe) unstable when a bin is empty in one distribution but not the other.

Attributes

class_representations_	Per-class histogram vectors.
block_slices_	Per-feature slices (when <code>partition_blocks=True</code>).

Example


```
>>> from mlquantify.representations import HistogramRepresentation
>>> rep = HistogramRepresentation(bins=(8,)).fit(scores, y)
>>> rep.transform(scores[:10]).shape
(8,)
```

USER GUIDE · role & theory

Role & mechanism

Discretises a distribution into a probability-mass vector that can be linearly mixed by a candidate prevalence and compared with a histogram distance — the classic mixture-model representation.

$$H(S)_i = (\# \text{ values of } S \text{ in bin } i) / |S| \quad (\text{normalised PMF})$$

$$\text{mix}(\alpha) = \alpha * H(S+) + (1-\alpha) * H(S-)$$

Used by

DyS, HDy, HDx, GHDy, GHDx.

References

[1] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

KDERepresentation — Kernel density representation

`representation` | `from mlquantify.representations import KDERepresentation`

API · interface

Summary

Per-class kernel density estimate over instances/posteriors.

Description

Fits a kernel density estimator per class (bandwidth, kernel) and exposes per-class likelihoods of test points; the test-time representation is the raw vector. A smooth, multivariate alternative to histograms.

Signature & parameters

bandwidth	float — the smoothing radius of the density estimate and the single most important parameter. Too small and each class density spikes on its training points (over-fitting); too large and the class densities blur together, flattening the differences the quantifier relies on.
kernel	shape of the local bump placed on each training point; sets how mass decays with distance. Default 'gaussian'. - gaussian — smooth, infinite-support bell; the usual default. - tophat — flat box: equal weight inside the bandwidth, zero outside. - epanechnikov — parabolic, finite support; theoretically efficient. - exponential — exponential decay with no hard cutoff. - linear — triangular weight decreasing linearly to the bandwidth edge. - cosine — cosine-shaped weight over the bandwidth.

Attributes

class_representations_	Per-class fitted KernelDensity models.
-------------------------------	--

Example

```
>>> from mlquantify.representations import KDERepresentation
>>> rep = KDERepresentation(bandwidth=0.2).fit(X, y)
>>> rep.class_representations_[0]
KernelDensity(bandwidth=0.2)
```

USER GUIDE · role & theory

Role & mechanism

Models each class density continuously, avoiding the bin-sparsity problem of histograms on the multiclass posterior simplex.

$$\text{KDE}_i(x) = (1/n_i) \sum_{x_j \text{ in class } i} K_h(x - x_j)$$

$$\text{mixture}(x) = \sum_i p_i * \text{KDE}_i(x)$$

Used by

KDEyML, KDEyHD, KDEyCS, GKDEyML.

References

[1] Moreo, A., González, P., & del Coz, J. J. (2024). Kernel Density Estimation for Multiclass Quantification. *arXiv:2401.00490*.

DistanceRepresentation — Distance representation

representation | from mlquantify.representations import DistanceRepresentation

API · interface

Summary

Mean pairwise-distance descriptor of a sample to each class.

Description

Summarises a set of instances by its mean pairwise distances to each training class, yielding an (`n_classes`,) descriptor — the basis of the energy-distance objective.

Signature & parameters

metric the ground distance between instances, passed to `scipy cdist`; it defines what 'close' means for the energy objective. Default 'euclidean'.

- `euclidean` — straight-line L2 distance (default).
- `cityblock` — Manhattan / L1 distance; the choice used by EDy on predictions.
- `cosine` — 1 minus cosine similarity; scale-invariant, good for sparse vectors.

Attributes

class_representations_ Mean class-to-training distances.

X_train_, y_train_ Stored training data.

Example

```
>>> from mlquantify.representations import DistanceRepresentation
>>> rep = DistanceRepresentation(metric='cityblock').fit(X, y)
>>> rep.transform(X_test).shape
(n_classes,)
```

USER GUIDE · role & theory

Role & mechanism

Provides the cross-distance terms q and M that make up the energy-distance quadratic form, without binning or density estimation.

$$q_l = \text{mean } \delta(\text{class } l, \text{ test})$$

$$M_{ll'} = \text{mean } \delta(\text{class } l, \text{ class } l')$$

Used by

EDy (on predictions), EDx (on features).

References

[1] del Coz, J. J. (2022). UniOviedo(Team2) at LeQua 2022: Comparison of Traditional Quantifiers and a New Method Based on Energy Distance. *CLEF 2022*.

KernelMeanRepresentation — Kernel mean embedding

representation | from mlquantify.representations import KernelMeanRepresentation

API · interface

Summary

Mean embedding of a sample in a reproducing-kernel Hilbert space.

Description

Represents a set of instances by its kernel mean embedding (the column-wise mean feature map), exact under a linear kernel and approximate under non-linear kernels.

Signature & parameters

kernel	the kernel whose feature map the sample is embedded under; determines which moments of the distribution are captured. Default 'rbf'. - rbf — Gaussian kernel; universal, captures all moments (default). - linear — embedding is just the mean vector; captures only the first moment. - poly — polynomial kernel; captures moments up to its degree. - sigmoid — hyperbolic-tangent kernel.
gamma	scale of the rbf/poly/sigmoid kernel; None defaults to 1/n_features. Controls how quickly similarity decays with distance.
degree	polynomial degree (poly kernel only) — the highest moment matched.
coef0	independent term in the poly/sigmoid kernel; shifts the similarity floor.

Attributes

class_representations_	Per-class mean embeddings.
-------------------------------	----------------------------

Example

```
>>> from mlquantify.representations import KernelMeanRepresentation
>>> rep = KernelMeanRepresentation(kernel='rbf').fit(X, y)
>>> emb = rep.transform(X_test)
```

USER GUIDE · role & theory

Role & mechanism

Embeds a whole distribution as a single point in an RKHS, so distribution matching reduces to matching mean embeddings (the MMD objective).

$$\Phi_{\text{bar}}(S) = (1/|S|) \sum_{x \in S} \Phi(x) \quad (\text{mean embedding})$$

$$\text{match: } \sum_y \theta_{y\text{y}} \Phi_{\text{bar}_y} \sim \Phi_{\text{bar_test}}$$

Used by

MMD_RKHS.

References

[1] Iyer, A., Nath, S., & Sarawagi, S. (2014). Maximum Mean Discrepancy for Class Ratio Estimation: Convergence Bounds and Kernel Selection. *ICML*, 32.

PredictionRepresentation — Prediction representation

representation | from mlquantify.representations import PredictionRepresentation

API · interface

Summary

Posterior / label representation (soft mean or hard class-frequency).

Description

Maps posteriors or labels to either the class-mean posterior ('soft') or the one-hot class-frequency vector ('hard', equivalent to Classify-and-Count on the representation). Custom transforms and a nested representation are supported. `HardPredictionRepresentation` and `SoftPredictionRepresentation` are the fixed-mode variants.

Signature & parameters

method	how classifier outputs become the descriptor (ignored when <code>func</code> is given). <code>soft</code> — average the posterior vectors — the class-mean posterior (used by GPACC). <code>hard</code> — one-hot the argmax label and average — the class-frequency vector, i.e. Classify-and-Count (used by GACC).
average	if <code>True</code> return the descriptor averaged over instances; if <code>False</code> return the per-instance matrix (needed when a downstream representation does its own pooling).
func	optional callable <code>func(X, representation) -> Z</code> replacing the built-in transform; lets you plug a custom mapping.
representation	optional nested representation applied to the transformed output before returning — e.g. feed soft posteriors into a histogram.

Attributes

priors_ Empirical class proportions seen during fit.

Example

```
>>> from mlquantify.representations import PredictionRepresentation
>>> rep = PredictionRepresentation(method='soft').fit(proba, y)
>>> rep.transform(proba_test)
```

USER GUIDE · role & theory

Role & mechanism

Turns classifier outputs into the mean-posterior or class-frequency vectors that the constrained-regression methods match on the simplex.

```
soft: z = (1/n) sum_x P(.|x)          (mean posterior)
hard: z = (1/n) sum_x onehot(argmax P(.|x))  (= CC)
```

Used by

GACC (hard), GPACC (soft), FM, and compose-based matching.

References

[1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.

PART IV

Losses

Objective functions minimised by the solvers. A quantifier family is largely the (representation, loss, solver) triple it composes.

Losses

Losses are the objective functions minimised by the solvers. A quantifier family is largely defined by the (representation, loss, solver) triple it composes; the same loss is reused across methods.

DistanceLoss — Distribution distance loss

`loss` | from `mlquantify.losses` import `DistanceLoss`

API · interface

Summary

Symmetric distance between two probability distributions.

Description

Computes a histogram/distribution dissimilarity (`hellinger`, `topsoe`, `prob_symm`, `sqEuclidean`, `euclidean`) between a mixture and a target, optionally normalising both to valid distributions first.

Signature & parameters

distance	the dissimilarity minimised between the two distributions; default 'hellinger'. <code>hellinger</code> — bounded distance over sqrt-probabilities; the classic HDy choice. <code>topsoe</code> — symmetric information-theoretic distance; best general performer in DyS. <code>probsymm</code> — probabilistic symmetric chi-square distance. <code>sqEuclidean</code> — squared Euclidean distance between the vectors. <code>euclidean</code> — Euclidean (L2) distance — the square root of <code>sqEuclidean</code>.
normalize	if True, clip and renormalise both inputs to valid distributions first, so the distance is well defined even on raw counts.

Returns

loss float, the distance value.

Example

```
>>> from mlquantify.losses import get_loss
>>> loss = get_loss('topsoe')
>>> round(loss([0.3, 0.7], [0.5, 0.5]), 4)
0.0528
```

USER GUIDE · role & theory

Role & mechanism

The objective minimised by histogram mixture models: find the prevalence whose mixed histogram is closest to the test histogram under this distance.

$$\text{hellinger}(P, Q) = \sqrt{1 - \sum_i \sqrt{P_i Q_i}}$$

$$\text{topsoe}(P, Q) = \sum_i P_i \ln(2P_i / (P_i + Q_i)) + Q_i \ln(2Q_i / (P_i + Q_i))$$

Used by

DyS, HDy, HDx (and any selectable-distance matcher).

References

[1] Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A Framework for Mixture Models in Quantification. *AAAI*, 33, 4552–4560.

LeastSquaresLoss — Least-squares loss

`loss` | from `mlquantify.losses` import `LeastSquaresLoss`

API · interface

Summary

Squared Euclidean (L2) loss for constrained regression.

Description

Computes $\| \text{target} - M \cdot \text{mixture} \|^2$; with no mixing matrix M it reduces to $\| \text{target} - \text{mixture} \|^2$. The objective of the unified constrained-regression framework.

Signature & parameters

mixture	estimated prevalence vector.
target	target representation vector.
M	optional mixing matrix applied as $M @ \text{mixture}$.

Returns

loss	float, the squared norm.
-------------	--------------------------

Example

```
>>> from mlquantify.losses import get_loss
>>> loss = get_loss('least_squares')
>>> loss([0.4, 0.6], [0.5, 0.5])
0.02
```

USER GUIDE · role & theory

Role & mechanism

Implements $y = X p$ in least-squares form: minimising it on the simplex inverts the per-class transform matrix to recover the prevalence.

$$L(p) = \| y - X p \|^2$$

minimised over $p \geq 0$, $\sum p = 1$

Used by

GACC, GPACC, FM, MMD_RKHS.

References

[1] Firat, A. (2016). Unified Framework for Quantification. *arXiv:1606.00868*.

HellingerSurrogateLoss — Hellinger surrogate loss

`loss` | `from mlquantify.losses import HellingerSurrogateLoss`

API · interface

Summary

Gradient-friendly surrogate for the squared Hellinger distance.

Description

Returns $-\sum_i \sqrt{p_i q_i}$; minimising it is equivalent to minimising the squared Hellinger distance but is numerically better behaved for gradient-free solvers.

Signature & parameters

mixture, target	the two vectors to compare.
M	optional mixing matrix.
normalize	normalise before computing.

Returns

loss	float; lower (more negative) is better.
-------------	---

Example

```
>>> from mlquantify.losses import get_loss
>>> loss = get_loss('hellinger_surrogate')
>>> round(loss([0.3, 0.7], [0.5, 0.5]), 4)
-0.9747
```

USER GUIDE · role & theory

Role & mechanism

Lets the multiclass Hellinger matchers be optimised on the simplex with SLSQP by dropping the constant and square-root-of-sum that make the raw Hellinger awkward.

$$H^2(p, q) = 1 - \sum_i \sqrt{p_i q_i}$$

$$\operatorname{argmin} H^2 == \operatorname{argmin} (- \sum_i \sqrt{p_i q_i})$$

Used by

GHDy, GHDx, KDEyHD.

References

[1] González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class Distribution Estimation Based on the Hellinger Distance. *Information Sciences*, 218, 146–164.

EnergyLoss — Energy-distance loss

`loss` | `from mlquantify.losses import EnergyLoss`

API · interface

Summary

Quadratic energy-distance objective for distribution matching.

Description

Computes the quadratic form $p^T(2q - M p)$, where q is the cross-distance vector (class-to-test) and M the pairwise class energy-distance matrix. Minimising it minimises the energy distance between the mixture and the test distribution.

Signature & parameters

prevalence	estimated prevalence p .
q	mean class-to-test distances.
M	pairwise class-to-class distance matrix.

Returns

loss	float, the energy-distance value.
-------------	-----------------------------------

Example

```
>>> from mlquantify.losses import get_loss
>>> loss = get_loss('energy')
>>> loss([0.5, 0.5], [0.4, 0.6], [[0,1],[1,0]])
0.5
```

USER GUIDE · role & theory

Role & mechanism

The closed quadratic that the energy-distance quantifiers minimise on the simplex, with q and M precomputed from the distance representation.

$$L(p) = p^T (2 q - M p)$$

$$q_l = \operatorname{mean} \operatorname{delta}(\operatorname{class} l, \operatorname{test}) ; \quad M_{ll'} = \operatorname{mean} \operatorname{delta}(\operatorname{class} l, \operatorname{class} l')$$

Used by

EDy, EDx.

References

[1] del Coz, J. J. (2022). UniOviedo(Team2) at LeQua 2022: Comparison of Traditional Quantifiers and a New Method Based on Energy Distance. *CLEF 2022*.

NegativeLogLikelihoodLoss — Negative log-likelihood loss

`loss` | from `mlquantify.losses` import `NegativeLogLikelihoodLoss`

API · interface

Summary

Negative log-likelihood of mixture likelihoods (mean or sum).

Description

Computes $-\log p(x)$ elementwise and reduces by mean or sum. The companion `MixtureNegativeLogLikelihoodLoss` and `RegularizedMixtureNLLLoss` build the mixture likelihood from per-class likelihoods and add optional simplex-smoothness penalties.

Signature & parameters

likelihood	per-sample mixture likelihoods in (0, 1].
reduction	how the per-sample negative log-likelihoods are combined into one scalar. - mean — average over samples; comparable across test bags of different sizes (default). - sum — total over samples; matches the raw joint log-likelihood.

Returns

loss float, the reduced negative log-likelihood.

Example

```
>>> from mlquantify.losses import get_loss
>>> loss = get_loss('nll')
>>> round(loss(np.array([0.5, 0.5])), 4)
0.6931
```

USER GUIDE · role & theory

Role & mechanism

The likelihood objective behind EM and KDE-ML quantifiers: choose the prevalence that maximises the likelihood of the test bag under the mixture density.

$$L(p) = - \sum_{x \text{ in test}} \log(\sum_i p_i * \text{lik}_i(x))$$

`argmin L == maximum-likelihood prevalence`

Used by

EMQ, KDEyML, GKDEyML, MLPE.

References

[1] Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the Outputs of a Classifier to New a Priori Probabilities: A Simple Procedure. *Neural Computation*, 14(1), 21–41.